

深入理解编译系统： 一个 SysY 程序的意义、表示与抉择

郭嘉睿 (2413174) 王艺霖 (2410824)

编译系统原理上机大作业预备工作

Abstract. 编译系统不是把源程序一次性“翻译”为机器码的黑箱，而是一系列关于意义、表示与实现选择的交接。本文以一个长度有界的 SysY 整数截断点积为贯穿案例，采用纵向综述的方式观察同一项计算如何依次成为预处理记号、带类型语法、LLVM 中间表示 (LLVM Intermediate Representation)、优化后的控制流与数据流、RV64 指令、带符号和重定位的 ELF 对象，以及可装载的进程映像。每到一层，我们都回答三个问题：源程序的哪项意义必须保留；当前表示让什么信息变得显式、又舍弃了什么；在语义允许的候选中，工程成本为何促使编译器作出特定选择。由此，静态单赋值形式、数据流不动点、指令选择与调度、寄存器分配、应用二进制接口和重定位不再是孤立术语，而成为同一程序逐步提交实现决定的不同机制。随后，本文以 MLIR 的多方言共存和渐进式降低解释为何领域结构应尽可能保留，并结合 AscendNPU IR 讨论异构系统中的存储层级与主机-设备边界；最后把验证编译、等式饱和、反馈驱动优化和即时编译组织为几组长期存在的设计张力。本文主张：理解编译器的关键，不是记住工具名称，而是追问信息应保存多久、决定应在何处作出，以及正确性约束如何与性能、编译时间和生态兼容性共同塑造实现。

Keywords: 编译系统 · 程序表示 · 语义精化 · LLVM IR · RISC-V · ELF · MLIR

作者分工

Table 1. 两位组员的工作分工。

成员	主要贡献
郭嘉睿	贯穿案例与形式化主线设计；SysY、预处理、前端、LLVM IR 与优化机制的综述和阐释。
王艺霖	RV64、ELF、链接与装载机制的综述；MLIR/AscendNPU IR 与编译前沿调研；视觉设计与排版。
共同完成	章节架构、跨层语义映射、工程权衡分析、文献整理与全文审阅。

1 引言：编译器是一连串可选择的表示

在命令行输入一次 `clang program.c`，得到一个可执行文件，很容易让人把编译器想成一个“把高级语言翻译成机器语言”的黑箱。这个说法没有错，却遗漏

了真正有趣的部分：`a[i]` 何时由一个源语言表达式变成地址计算？循环变量何时不再占有栈槽，而成为一条数据流边？一条尚不知道目标地址的函数调用为什么已经能够写进目标文件？优化器又为何有权把程序改得面目全非？这些问题不能由一次“翻译”回答，因为它们发生在不同的表示中，受不同的规则约束，也保留不同种类的自由。

本文把编译系统（compiler system）理解为一个表示链：字符与记号、带类型的抽象语法树（abstract syntax tree, AST）、LLVM 中间表示、目标相关机器表示、汇编与可重定位目标文件，最后是被加载的进程。每跨过一个边界，都有某些事实变得显式，某些源级细节被有意忘记，还有一些决定继续推迟。例如，前端确认 `a[i]` 是一个整数左值，却不决定使用哪条寻址指令；LLVM IR 把地址和加载写出来，却不决定物理寄存器；汇编器把指令编码为字节，却把外部函数的地址保留成重定位。所谓表示契约（representation contract），正是相邻层关于“已经知道什么、仍可改变什么”的约定，而不是某个内部数据结构永不变化的 API。

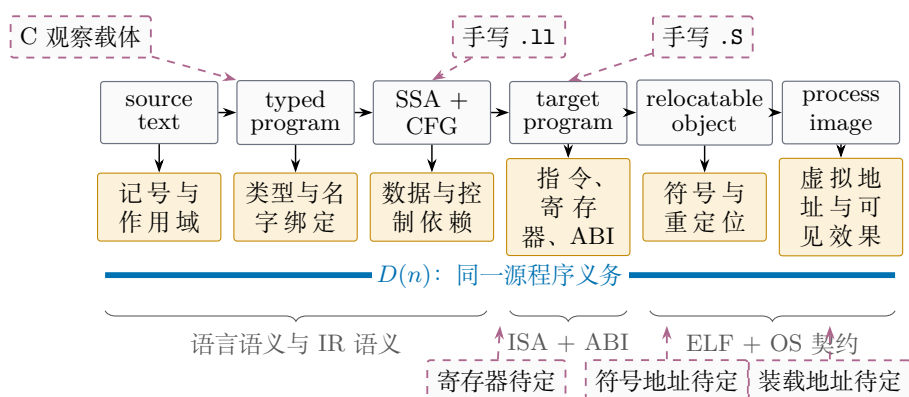


Fig. 1. 同一个计算在六种表示中拥有不同的显式结构。蓝色粗线表示必须保持的语义关系；橙色浅底标签表示本层新增的决定；紫色虚线表示仍被有意延期的决定。方框是表示边界，而不等同于操作系统进程。

这张全景图也纠正了一个常见误会：概念阶段不必等于操作系统进程。Clang 的驱动程序（compiler driver）构造动作图，再把动作绑定为实际工具与作业；预处理、前端、后端乃至集成汇编器可以在同一个 `clang -cc1` 作业中连续完成 [26,24]。因此，“预处理器—编译器—汇编器—链接器”是有用的责任分解，却不是对进程数量的承诺。理解编译系统，应当追踪表示与决定，而不是数命令行里启动了几个程序。

1.1 一条语义线索贯穿全文

我们只跟随一个小程序：它读入前缀长度 n ，在两个长度为 8 的整型数组上逐项相乘，把每个乘积限制在 $[-8, 12]$ ，然后求和输出。这个“有界前缀截断点积”包含数组、下标、可变状态、条件、循环、用户函数和外部运行时调用；它足够

小，可以逐块阅读，又足以让编译系统的大多数关键机制自然出现。案例不是等待被测试的研究对象，而是一根叙事线索：后文会不断指向同一个索引、同一个累加值和同一次 `putint` 调用，观察它们在每层如何改换载体。

“仍是同一个程序”也需要一个适度精确的说法。令 $\text{Beh}_L(P)$ 表示程序 P 在语言或表示 L 中允许的可观察行为 (observable behavior) 集合。对于从源表示 L_s 到目标表示 L_t 的降低 T ，理想化的语义义务是

$$\text{Beh}_{L_t}(T(P)) \subseteq \text{Beh}_{L_s}(P). \quad (1)$$

这里用包含而非朴素相等，是因为源语言可能允许多种求值顺序或实现选择；目标程序可以选定其中一种，却不应为定义良好的源程序制造源语义不允许的结果。这种关系称为语义精化 (semantic refinement)。式 (1) 依赖源程序无未定义行为、ABI 与运行环境匹配等前提，也不声称本文对 LLVM 完成了形式化证明。它的用途更朴素：把“意义必须保留”与“表示可以自由选择”分开。对本文的确定性整数案例，允许的核心观察就是读入一个整数并输出相应的和；后续每一层都在实现这个函数，同时逐步兑现布局、指令和地址方面的选择。

这一区分揭示了编译器设计的两类判断。第一类是合法性：某种变换是否仍满足语义精化；第二类是收益性 (profitability)：在众多合法结果中，哪一个更符合运行时间、代码大小、编译时间或能耗目标。数学模型适合划出不可越过的边界，却不会自动给出生产系统中的唯一答案。真实编译器必须在不完整的别名信息、有限的编译预算、变化的微架构和稳定 ABI 之间作出工程折中。

1.2 阅读每一层的三个问题

全文围绕三个问题推进，而不是围绕工具选项罗列材料：

1. **意义**：此层必须保住案例中的哪一个事实？
2. **表示**：该事实如何编码，什么变得显式，什么已被丢弃或抽象？
3. **选择**：理论条件排除了哪些变换，工程代价又如何在剩余方案中作选择？

例如，前端通过名字解析和类型推导确认数组下标有意义；LLVM IR 把循环拆成基本块并以 静态单赋值 (static single assignment, SSA) 值表达迭代间的数据联系；后端把语义自由压缩成 RV64 的指令、寄存器和调用约定；汇编、链接与加载则逐步把符号关系兑现成进程地址。每章都会先给出足以解释机制的形式化骨架，再立即回到案例，说明这个模型在工程实现中获得了什么、又付出了什么。

本文使用 SysY 2022 作为规范源语言。为了观察成熟 Clang 前端，还提供一个只覆盖本案例兼容子集的 C17 载体；它是教学用观察窗口，不把 Clang 宣称为完整 SysY 前端。手写 LLVM IR 与 RV64 汇编则让我们能从相应层次开始阅读同一计算。四种形式在代表性边界输入上得到相同输出，这满足课程的实现检查；具体命令与运行矩阵留在配套工程中，不让构建日志打断概念叙事。

1.3 文章结构与中心命题

第 2 节先定义计算本身，并画出同一事实的多种形态；第 3 节解释记号、语法、名字和类型怎样把字符变成“有意义的程序”。后续章节依次进入 LLVM IR、优化、RV64 代码生成、ABI、ELF、链接与加载，再以 MLIR 和 AscendNPU IR

说明为何现代编译器需要让领域抽象存活得更久，最后讨论验证、搜索、反馈优化与动态编译等前沿方向。

本文不提出新的编译算法。它试图建立一种可迁移的阅读方法：不要把编译阶段当成工具清单，而把它们看作语义责任的交接。一个好的表示不会保存所有信息；它保存下一组决策真正需要的信息，并把尚未成熟的决定推迟到更了解约束的地方。沿着这条原则，我们可以从一行 SysY 代码一直读到进程映像，而不把任何一层误认为整个编译器。

2 一个计算，多种形态：有界前缀截断点积

在研究编译器怎样改变程序之前，必须先说清程序做什么。本章不讨论寄存器、基本块或目标文件，而把案例压缩成一个数学函数和几条不变量。它们是后续所有表示共同承担的语义线索。

2.1 规范源程序

代码清单 1.1 是全文唯一的源级案例。SysY 运行时预先提供 `getint`、`putint` 与 `putch`；数组形参传递首元素地址，标量形参按值传递。这一区别看似只是语法规则，后来会分别成为 LLVM 的指针/整数参数以及 RISC-V ABI 中的寄存器值。

Listing 1.1. 规范性 SysY 案例。

```
int clipped_dot(int a[], int b[], int n, int lo, int hi) {
    int acc = 0;
    int i = 0;
    while (i < n) {
        int term = a[i] * b[i];
        if (term < lo) {
            term = lo;
        } else {
            if (term > hi) {
                term = hi;
            }
        }
        acc = acc + term;
        i = i + 1;
    }
    return acc;
}

int main() {
    int a[8] = {-4, 1, 7, 3, 9, -2, 6, 5};
    int b[8] = { 2,-3, 1, 4,-1, 5, 2, 3};
    int n = getint();
    if (n < 0) {
        n = 0;
    } else {
```

```

    if (n > 8) {
        n = 8;
    }
    putint(clipped_dot(a, b, n, -8, 12));
    putchar(10);
    return 0;
}

```

令

$$\text{clip}(x, l, h) = \min(h, \max(l, x)), \quad k = \text{clip}(n, 0, 8).$$

程序实现的函数为

$$D(n) = \sum_{i=0}^{k-1} \text{clip}(a_i b_i, -8, 12), \quad (2)$$

其中空和等于 0。两个固定数组的乘积

$$(-8, -3, 7, 12, -9, -10, 12, 15)$$

经逐项截断变为

$$(-8, -3, 7, 12, -8, -8, 12, 12),$$

所以长度 0 到 8 的前缀和依次是 0, -8, -11, -4, 8, 0, -8, 4, 16。这些数字的主要作用不是构造一个庞大的测试仪式，而是让读者随时能够把抽象机制重新落到一个可心算的计算上。

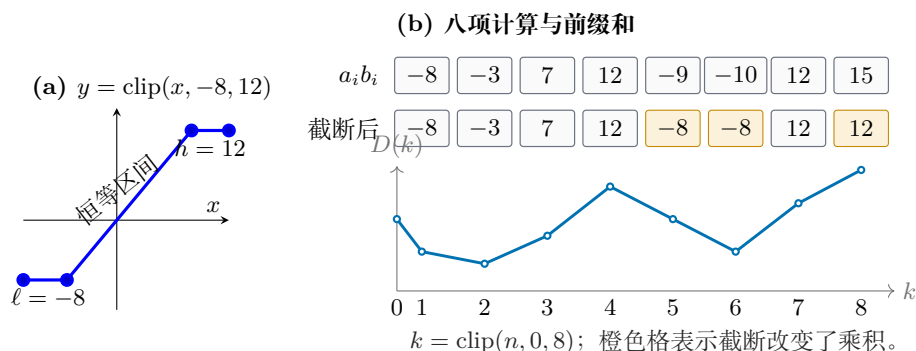


Fig. 2. 案例计算的两次限制发生在不同对象上：输入决定前缀长度 k ，逐项截断决定每个加数；蓝色折线给出 $k = 0, \dots, 8$ 时的前缀和。

图中的同一乘法在各层换了身份：源程序里它是两个下标表达式；带类型 AST 中还包含隐式的左值到值转换；LLVM IR 把元素地址、加载与乘法拆开；RV64 用地址算术、`lw` 和 `mulw` 实现；目标文件只保存编码后的字节和必要的符号关系。跨层延续的不是变量名或排版，而是“取两个第 i 个整数并相乘”这条数据依赖。

2.2 三条不变量连接函数与循环

案例的第一个不变量是内存安全：

$$0 \leq i < k \leq 8.$$

输入先被限制到 $[0, 8]$ ，循环条件成立后才会求值数组下标，因此每次访问都落在两个数组内部。这让后文可以专注于地址是怎样表示和优化的，而不必把偶然越界行为混入讨论。

第二个不变量把命令式循环连接到式 (2)。记 $c_j = \text{clip}(a_j b_j, -8, 12)$ 。在闭合的 `main` 调用中，传给 `clipped_dot` 的长度正是 k ；循环头应满足

$$I(i, acc) \equiv 0 \leq i \leq k \wedge acc = \sum_{j=0}^{i-1} c_j. \quad (3)$$

初始化时 $i = 0, acc = 0$ ，空和使 $I(0, 0)$ 成立；若 $I(i, acc)$ 成立且 $i < k$ ，一次循环执行 $(i, acc) \mapsto (i + 1, acc + c_i)$ ，于是新状态仍满足 I ；退出时条件否定，结合 $0 \leq i \leq k$ 得到 $i = k$ ，因而 $acc = D(n)$ 。这三个步骤不是为了把文章改写成证明报告，而是给后文一根真正守恒的轴：SysY 的两个赋值、SSA 循环头的两个 `phi`、RV64 的回边以及 MLIR 的归约迭代参数，都在编码同一个状态转移。

第三个不变量是整数范围。原始乘积落在 $[-10, 15]$ ，截断后的每项落在 $[-8, 12]$ ，八项和必在 $[-64, 96]$ 。因此本例的 32 位有符号乘法和加法不会溢出。这个范围证明虽短，却有直接的工程后果：优化器无需借助有符号溢出的未定义行为来解释本例，读者也可以把源级整数与 LLVM `i32` 运算对应起来。若数组改为任意用户输入，同一实现就需要更宽的中间类型、显式溢出策略或更严格的前置条件；追求更一般的接口，会把额外语义成本传到每一层。

逐项截断还揭示了一个容易被“点积”名称遮蔽的区别。这里先计算每个 $\text{clip}(a_i b_i, -8, 12)$ ，再用普通加法归约；由于和不会溢出，改变加法树的形状不会改变结果。若改为每次更新累加器后都做饱和截断，混合正负项会破坏结合性，树形归约和向量化便不能直接保持顺序语义。编译器从不优化“看起来像点积”的东西；它优化满足特定代数条件的运算。

2.3 为何一个小程序足以打开整个系统

案例的每个结构都向后投下一道影子：

- `a[i]` 与 `b[i]` 要经过类型检查，随后变成元素地址、加载以及目标机寻址；
- `i` 与 `acc` 在未优化表示中可能位于栈槽，在 SSA 中则成为循环携带值，最终竞争有限的物理寄存器；
- 两级截断条件先成为控制流图中的分支与汇合，也可能被优化成选择或最值序列；
- 五参数函数迫使后端服从调用约定，即使另一种局部寄存器安排看上去更漂亮；
- `putint` 从带类型调用变为外部符号和重定位，直到链接时才获得定义。

这个选择体现了一条写作原则：案例无需模拟真实应用的规模，只需让重要机制不可回避。加入递归、浮点或并发会产生新的叙事线，却不会使当前这条线解释得更清楚。

2.4 四种材料，而非四个彼此竞争的程序

后文会看到四种材料。规范 SysY 源文件给出源级含义；C17 观察载体仅为借助成熟 Clang 展示预处理、记号与 AST，在本案例的整数子集内镜像同一算法；Clang 生成的 IR 和汇编展示生产编译器作出的选择；手写 LLVM IR 与 RV64 汇编则让我们从中间层直接阅读同一计算。它们服务于同一讲解，却承担不同角色，不能把“Clang 接受这份 C”误写成“Clang 是一个完整 SysY 前端”。

课程实现还把这四种形式在 $n = -1, 0, 1, 4, 8, 10$ 上运行，所得输出依次为 0, 0, -8, 8, 16, 16。这一小表足以确认配套产物可以实际汇编、链接和运行；它不是文章的中心，更不替代对表示与机制的解释。确切命令、工具版本和运行时诊断保存在 preflight/README.md 中。

2.5 路线图：每一层得到什么，又放弃什么

从源程序向机器前进，并非给每个变量不断换一种拼写。前端保留声明、类型和源位置，适合诊断，却不适合全局数据流优化；LLVM IR 放弃大部分源语法，以基本块、显式内存操作和 SSA 关系换取跨语言优化能力；机器表示接受寄存器类别、指令延迟和 ABI 约束，换取可执行性；ELF 又把已确定的字节与尚未确定的地址方程一起封装，换取独立编译和部署弹性。

因此，本教程式旅程的核心并不是“源代码经过了多少步骤”，而是三个连续问题：当前表示让哪种推理变得容易？它不再保存什么？被推迟的决定将由谁完成？下一章从第一个边界开始：字符与宏本身并不知道 `a[i]` 合不合法，前端必须把线性记号构造成带绑定、类型和值类别的程序。

3 意义先于代码生成：预处理与编译前端

源文件中的字符并不知道 `clipped_dot` 是函数、`a[i]` 是可读取的整数对象，也不知道 `-8` 中的负号是运算符。代码生成之前，编译器必须先回答一个更根本的问题：这些记号能否组成一个有意义的程序？前端把字符逐步组织成记号、语法结构、名字绑定和带类型操作；它产出的不是一棵漂亮的括号树，而是一份足以承担后续降低的语义结构。

3.1 两个入口：规范 SysY 与 C 观察窗口

规范 SysY 源文件不含 `#include` 与宏，因为 SysY 语言本身不定义 C 预处理器，运行时函数由语言环境提供。为了观察成熟工具的完整前端路径，配套工程另有一个 C17 载体，其中包含：

```
#include "case_config.h"

/* case_config.h */
#define VECTOR_LENGTH 8
#define TERM_MIN (-8)
#define TERM_MAX 12
int getint(void);
void putint(int value);
void putch(int value);
```

三个宏与运行时声明让 Clang 能在本案例的 C/SysY 兼容子集上展示预处理和 AST。两份源码并非两套规范：SysY 文件定义本文要解释的程序，C 文件只是观察窗口。它额外使用 `const`、`main(void)` 和 `else if` 等 C 写法，但这些差异不改变式 (2) 描述的计算。把这个边界说清一次，后文便不必反复用工具来源打断机制讲解。

3.2 预处理：在类型出现之前改写记号

C 的翻译阶段 (translation phases) 规定一种概念顺序：字符映射和行拼接之后，源文本被识别为预处理记号，注释被空白取代；随后处理包含、条件指令和宏展开，再把结果交给语言的语法与语义分析 [17]。这是语义次序，不要求实现真的多次扫描文件。GNU CPP 会为性能融合早期步骤；Clang 的词法器、预处理器和解析器也以流式方式协作 [13,25]。理论分层回答“谁负责什么”，实现融合则减少中间物化、提高局部性；二者并不矛盾。

预处理按记号而非任意字符串工作。`VECTOR_LENGTH` 被对象式宏替换为 `8`；`TERM_MIN` 被替换为记号序列 `( - 8 )`。此时预处理器既不知道它们将用于数组维度，也不知道结果类型是 `int`。直到后续语法把 `-` 解释为一元运算，语义分析才把整个表达式定为整数。宏因而具有很低的语义门槛：它快速、历史兼容、能在解析前适配平台，却无法借助语言类型保证替换结果合理。

`#include` 也不只是一次盲目的文本粘贴。实现根据引号形式、`-I`、系统目录和目标 `sysroot` 定位文件，扫描其中的指令和记号，再返回原输入。包含卫士通过条件编译让头文件内容在同一翻译单元中只进入一次。被 `#if 0` 排除的分支甚至无须构成合法 C 语法，因为解析器永远看不到它；而案例中的 `if (term < 10)` 必须进入 AST 并成为运行时选择。条件编译以牺牲局部可见性为代价，把构建配置变成程序的一部分，这正是大型 C/C++ 工程既依赖它又努力约束它的原因。

3.3 从记号序列到语法结构

词法分析 (lexical analysis) 把字符组织成带种类、拼写和源位置的记号。循环中的 `term = a[i] * b[i]`；形成如下核心序列：

$$id(term), =, id(a), [, id(i),], *, id(b), [, id(i),], ;.$$

在这一层，`while` 已是关键字，`<=` 与 `< =` 已有不同记号划分，但 `term` 是否已声明仍无从判断。类似地，`-8` 通常是运算符 `-` 和常量 `8` 两个记号，而不是一个“负八”记号。

语法分析 (parsing) 依据文法把线性序列组织成树。抽取与案例有关的三个产生式，可写成

$$\begin{aligned} E &::= E[E] \mid E * E \mid E + E \mid id \mid integer, \\ S &::= while(E) S \mid if(E) S else S \mid E;, \\ B &::= \{S^*\}. \end{aligned}$$

运算符优先级使 `a[i] * b[i]` 先构造两个下标节点，再构造乘法节点；“悬空的 `else`”规则把 `else` 与最近未配对的 `if` 结合。真实 Clang 采用递归下降解析，并在识别结构时调用 Sema，而不是先造一棵完整的纯语法树再进行完全独立的一遍检查 [25]。责任上区分 parsing 与 semantic analysis 依然重要，实现上交错二者则能改善错误恢复和上下文相关解析。

3.4 类型判断：为什么 $a[i] * b[i]$ 有意义

令类型环境 Γ 把名字映射到类型。本例函数体中有 $\Gamma(a) = \Gamma(b) = \text{int}^*$ 、 $\Gamma(i) = \text{int}$ 。用 $\text{lval}(\tau)$ 表示可定位、尚待读取的 τ 类型对象，下标与读取规则可以压缩为

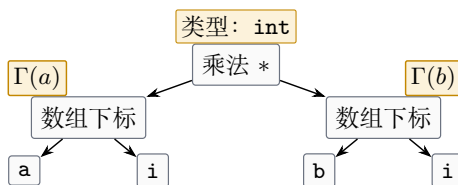
$$\frac{\Gamma \vdash a : \text{int}^* \quad \Gamma \vdash i : \text{int}}{\Gamma \vdash a[i] : \text{lval}(\text{int})} \quad \frac{\Gamma \vdash e : \text{lval}(\tau)}{\Gamma \vdash \text{load}(e) : \tau}.$$

再结合整数乘法规则，可得到一个真正的推导：

$$\frac{\Gamma \vdash a[i] : \text{lval}(\text{int}) \quad \Gamma \vdash b[i] : \text{lval}(\text{int})}{\Gamma \vdash \text{load}(a[i]) * \text{load}(b[i]) : \text{int}}. \quad (4)$$

源代码没有写 `load`；C/Clang AST 以隐式的左值到右值转换表达它。数组实参传给 `clipped_dot` 时则发生数组到指针调整，而不是复制八个整数。式 (4) 说明了解析树为何不够：相同形状的 $x[y]$ 只有在基址和索引满足类型前提时才有程序意义，也只有这样，下一层才有理由生成元素地址与 32 位加载。

(a) 结构与绑定



(b) 类型判断

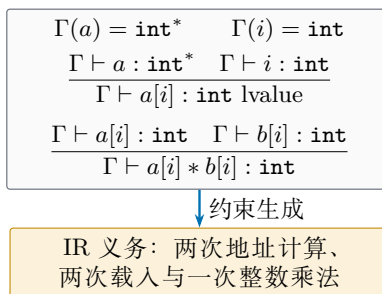


Fig. 3. 解析给出表达式的树形结构，语义分析把名字、类型与隐式转换附着到同一结构；这些判断随后约束地址计算、载入和乘法的 IR 生成。

丰富的类型和源位置信息使前端能够给出精确诊断，也能把隐式操作明确交给 IRGen；代价是 AST 高度依赖源语言，节点繁多，维护和遍历成本较高。反过来，若过早压平为少数通用节点，后端接口会更简单，却很难告诉用户究竟是哪一个源级规则失败。生产前端因此宁可在 AST 中保存比优化器需要更多的信息，再在降低时有选择地丢弃。

3.5 名字、作用域和值类别

语义分析 (semantic analysis) 还负责把标识符绑定到声明。函数形参 `a, b, n, lo, hi` 与局部变量 `acc, i, term` 各有声明实体和作用域；循环体中的每个 `i` 都解析到同一个局部声明，`term` 则只在相应复合语句内可见。若在声明之前引用一个名字，字符与语法可以完全合法，名字查找仍会拒绝程序。符号表

因而不只是“字符串到地址”的字典，而是作用域、声明类别、重声明关系和类型信息共同参与的结构。

值类别进一步决定表达式能做什么。`a[i]` 是可赋值对象的左值 (lvalue)；在乘法中需要其数值，所以发生读取；在赋值号左侧则保留位置含义。`term < lo` 的两个操作数被读取为整数，比较结果可作为条件；`putch(10)` 必须解析到接收一个整数且无返回值的运行时声明。前端在这里确立调用类型和副作用边界，却不指定参数最终位于 `a0` 还是栈上，因为那是目标 ABI 才掌握的决定。

常量求值也不同于宏展开。规范源中的 8 可在编译期证明为合法数组维度；C 载体中的 `VECTOR_LENGTH` 要先展开成同一个字面量，再接受带类型的常量表达式规则。前者属于语言语义，后者只是解析前的记号改写。把两者混为一谈，会让人误以为预处理器理解数组类型。

3.6 诊断：前端的用户界面

考虑把调用改成 `clipped_dot(a,b,n,-8)`。词法器仍可产生全部记号，解析器也能构造四实参调用；只有名字解析找到五参数声明并进行参数匹配后，Sema 才能指出实参数量不足。同样，`a[1.5]` 可以形成下标语法，却违反整数索引约束。诊断并非编译器之外的附属文本，而是前端无法建立带类型表示时面向程序员的结果。

这里存在一个典型工程折中。为了在第一个错误后继续报告更多问题，生产前端常构造含无效节点的局部 AST 并进行错误恢复；若坚持“任何不完美输入立即终止”，实现更简单，用户体验却很差。恢复过度又可能产生级联误报。好的前端在保持内部不变量与保留足够上下文之间取平衡，而不是把形式文法的拒绝/接受二分直接当成完整产品行为。

3.7 前端的终点：已解释，但尚未布局

到带类型 AST 为止，前端已经知道数组元素是整数、比较采用有符号语义、循环存在一条结构化回边，运行时调用具有特定函数类型。它仍不知道数组在栈帧中的偏移、循环值占用哪个寄存器、外部调用位于哪个地址；这些信息既依赖目标，也尚无必要决定。Clang IRGen 因而把结构化控制拆成基本块和分支，把左值访问拆成地址与加载/存储，把函数调用变成带类型 LLVM 调用 [20]。

这种降低有意丢失宏边界、`else if` 的原始拼写和许多源级类别，同时让控制流、数据流与内存操作变得可以统一分析。前端证明了 `a[i]` 是什么；下一层必须回答它怎样成为地址，以及循环中的 `i` 和 `acc` 如何在多条控制流边之间保持身份。这正是控制流图、支配关系与 SSA 出场的理由。

4 LLVM IR：把循环拆成控制、值与内存

前端已经知道 `a[i]` 是一个整数表达式，却仍未回答三个问题：什么时候执行这次访问？本轮的 `i` 与 `acc` 来自哪里？数组下标怎样成为地址？LLVM 中间表示 (LLVM Intermediate Representation, LLVM IR) 的作用正是把这些隐含在结构化语句里的关系分开表达。它既不是源语言的简写，也不是某一种机器的汇编，而是一层有类型、显式控制流且便于改写的语义接口 [21,44]。

本节继续追踪 `clipped_dot`：循环头决定是否处理第 i 项，循环体读取 a_i, b_i ，计算并截断乘积，再把结果并入 `acc`。同一个小循环将依次显露为一张控制流图、两条跨迭代的值递推和两次带类型步长的地址计算。

4.1 基本块组成控制流图

函数首先被划分为基本块（basic block）：除入口外只能从块首进入，块内顺序执行，并由 `br`、`ret` 等终结指令决定下一步。令

$$G = (V, E), \quad V = \{entry, loop, body, exit\}.$$

其中边 $(u, v) \in E$ 表示块 u 的终结指令可能把控制交给 v 。案例有入口边 $entry \rightarrow loop$ 、真假分支 $loop \rightarrow body/exit$ 和回边 $body \rightarrow loop$ 。源语言的 `while` 因而不只是一个特殊构造，而是图上的一个环。

图 4 把这张图与循环携带值放在一起。它比源代码多暴露了分支边，却有意忘掉“`while` 语句”这种表面结构；这种取舍使同一套图算法可以处理由 `while`、`for`、短路表达式乃至 `goto` 形成的控制流。

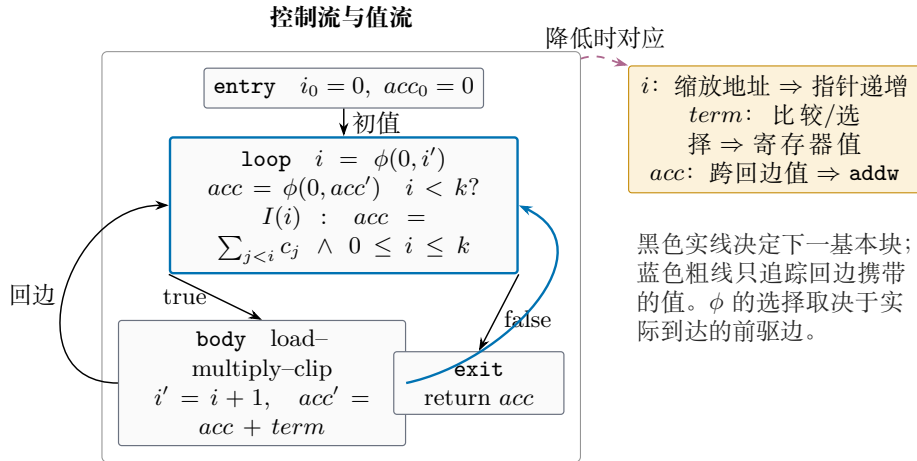


Fig. 4. 循环同时是一张控制流图和一组跨回边的数据依赖： ϕ 节点表达“来自哪条前驱边的值”，而后端可以把索引归纳改写为指针递增，只要这些依赖保持不变。

图结构还给出了“某个定义能否到达某次使用”的精确语言。若从入口到结点 v 的每一条路径都经过 d ，称 d 支配（dominate） v ：

$$d \text{ dom } v \iff \forall \pi : entry \rightsquigarrow v, d \in \pi. \quad (5)$$

例如 $loop$ 支配 $body$ 和 $exit$ ， $body$ 却不支配 $loop$ ，因为第一次迭代从 $entry$ 直接进入循环头。LLVM 要求普通 SSA 定义支配其使用；支配关系由此把“先定义后使用”从文本顺序推广到了任意 CFG。

工程上当然可以保留结构化语法树并在其上优化，但大量语言的控制结构会迫使每个分析分别处理许多语法特例。CFG 牺牲一部分源级意图，换来统一的可达性、循环和支配算法。调试信息与后续的循环元数据，正是在这种扁平化之后补回高层来源和意图的工程机制。

4.2 SSA 与 phi：在边上选择值

静态单赋值（Static Single Assignment, SSA）要求每个 SSA 名字恰好由一处定义。案例的循环头为

```
loop:
  %i = phi i32 [ 0, %entry ], [ %i.next, %body ]
  %acc = phi i32 [ 0, %entry ], [ %acc.next, %body ]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %exit
```

这里 phi 不是按文本顺序执行的普通赋值。若控制沿边 $p \rightarrow loop$ 到达，则所有 phi 同时选择标有前驱 p 的来值：

$$(i, acc)_{loop} = \begin{cases} (0, 0), & p = entry, \\ (i_{next}, acc_{next}), & p = body. \end{cases}$$

第一次进入循环时取初值；走回边时取上一轮产生的新值。phi 的使用概念上位于对应前驱边上，因此 %i.next 无须支配循环头，只须在 $body \rightarrow loop$ 这条边上可用。这一边语义也避免了把两个 phi 错解成有先后次序的交换赋值。

SSA 的收益不只是“变量不再改名”。一个值只有唯一来源，常量传播、公共子表达式消除和许多稀疏数据流分析便可直接沿 def-use 链工作；在汇合点需要 phi 的位置则可由支配边界系统构造 [9]。然而 SSA 只约束值名，并没有消除可变内存：

```
%slot = alloca i32, align 4
store i32 %x, ptr %slot, align 4
%y = load i32, ptr %slot, align 4
```

%y 只定义一次，但 %slot 指向的对象可以被反复写入。对地址未逸出、访问形状简单的局部对象，内存提升（memory promotion）可以把 load/store 改成 SSA 值，并在必要的汇合点插入 phi [46]；数组、别名指针、volatile 访问和未知调用则需要另外的内存效应模型。理论上更纯粹的“所有内存皆 SSA”会把每次存储都变成显式状态版本，实际系统通常以 MemorySSA、别名分析和效应属性分工，以控制表示规模与编译时间。

4.3 GEP：类型化地址计算，而不是访存

循环体把数组访问拆成索引扩展、地址计算和读取：

```
%index = sext i32 %i to i64
%a.element = getelementptr inbounds i32, ptr %a, i64 %index
%a.value = load i32, ptr %a.element, align 4
```

在本例默认地址空间中，忽略实现规定的指针位宽细节后，这条 getelementptr (GEP) 的核心地址关系为

$$\text{addr}(a[i]) = \text{base}(a) + \text{sext}_{64}(i) \cdot \text{allocsize}(i32) = \text{base}(a) + 4i. \quad (6)$$

这正是前端类型判断留下的最后一项高层信息：`i32` 决定步长；GEP 本身不读取内存，随后的 `load i32` 才产生读效应。现代 LLVM 的不透明指针（opaque pointer）不在 `ptr` 上重复保存所指类型，访问类型由 GEP、load 和 store 明示 [48,47]。

`inbounds` 不是边界检查。它承诺地址计算保持在同一已分配对象允许范围内；承诺被违反时结果可成为 `poison`。案例的 `main` 将 n 夹到 $[0, 8]$ ，循环体只在 $0 \leq i < n$ 时执行，两个实参数组恰有 8 项，所以当前调用满足承诺。但是 `clipped_dot` 的独立函数签名没有编码“至少 8 项”这一长度关系：若希望对任意调用者保持该性质，就需要更强的接口、调用点内联事实，或放弃 `inbounds`。更强的 IR 承诺扩大优化空间，也同步扩大了前端必须正确证明的义务。

4.4 属性、内存与 `poison` 的边界

函数参数写作 `ptr readonly %a`，只说明该函数不经这个参数写入可达内存；它不自动意味着非空、足够长、互不别名或不会逸出。类似地，数据布局（data layout）决定指针宽度、标量对齐和聚合体步长，目标三元组（target triple）选择目标环境。模块仍可跨多个后端复用，但并非脱离目标约束的纯数学项 [41]。

这些契约之所以重要，可由一个局部变换看出。普通 `add i32` 定义模 2^{32} 的位向量加法；`add nsw` 则额外承诺有符号加法不环绕，违反时产生 `poison`。`poison` 通常沿数据流传播，一旦被用于分支条件或地址等敏感位置即可导致未定义行为；`freeze` 能把 `poison/undef` 固定为一次任意但一致的普通值 [44]。因此 `nsw` 不是免费的优化提示，也不是动态检查，而是允许优化器排除溢出执行的语义前提。

在固定案例中，每项截断后落于 $[-8, 12]$ ，八项和落于 $[-64, 96]$ ，所以累加不会溢出；但截断前的乘法若面对任意 `i32` 数组，仍可能溢出。手写 IR 对乘法使用普通 `mul`，而从 C 生成的 IR 可依据“有符号溢出为未定义行为”添加 `nsw`。两者在当前数据上得到同一结果，却为一般调用定义了不同的程序。这不是需要反复附加的免责声明，而是下一节判断优化是否合法时必须区分的语义事实。

本层原则。LLVM IR 的力量来自分离：CFG 表达何时执行，SSA 表达值从何处来，GEP 表达地址如何形成，内存效应与属性表达哪些重排仍被允许。分离让分析能够各自处理一种关系，也意味着优化器必须重新组合控制、数值与内存事实；下一节的核心问题因此不是“能否写出另一段 IR”，而是“哪一种改写既合法又值得”。

5 优化：先证明可以改，再判断值得改

把三路截断控制流写成 `select`、把标量循环改成向量循环，看起来都像“更快的写法”。但优化器必须回答两个彼此独立的问题：改写是否保留源程序承诺的行为，以及在特定目标和工作负载上是否比原形状更划算。前者是合法性（legality）的硬约束，后者是盈利性（profitability）的工程判断。图 5 将它们画成前后两道门；不能通过第一道门的候选，无论估计性能多高都不是优化。

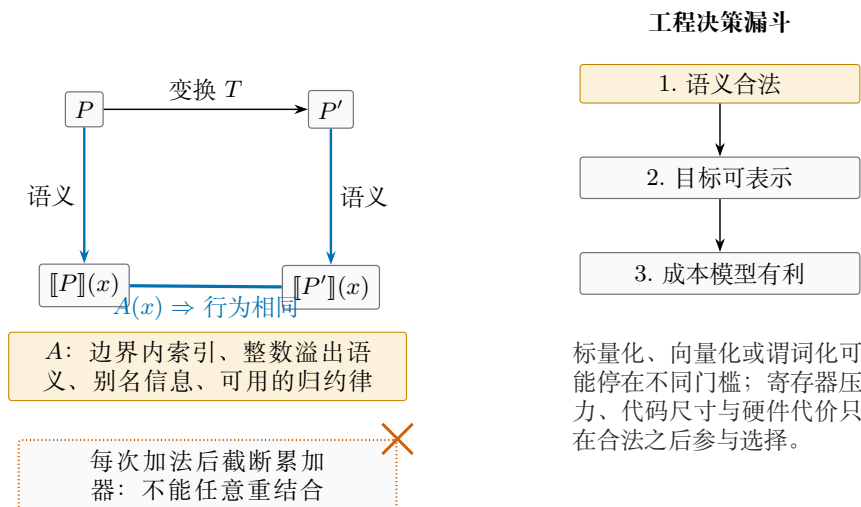


Fig. 5. 优化首先要求语义图在假设 A 下交换；通过合法性门槛后，目标特性与成本模型才决定是否采用该改写。合法不等于有利，未采用也不等于不可证明。

5.1 分析是关于程序图的不动点计算

优化并非先遍历文本，再凭模式猜测。分析通常在一个抽象域 $(L, \sqsubseteq)$ 中为每个基本块求事实。以正向“必须成立”分析为例，块 B 的入口事实来自所有前驱的合流，块内语句由单调递函数 F_B 更新它：

$$IN_B = \bigwedge_{P \in \text{pred}(B)} OUT_P, \quad OUT_B = F_B(IN_B). \quad (7)$$

对 CFG 反复应用这些方程，直至事实不再改变，就得到一个不动点 (fixed point)。在有限高度格上单调迭代会终止；区间等无限域通常需要 widening 等近似手段。这里的“抽象”很具体：常量传播的格元素可表示未知、某个常量或冲突；区间分析保存上下界；别名分析保存两个访问必定、可能或不可能重叠。

在 `clipped_dot` 中，`loop` 同时接收入口边与回边。常量传播第一次看到 $i = 0$ ，回边又带来 $i + 1$ ，很快便只能把 i 记为“非常量”；区间分析则可能保留 $i \geq 0$ ，并在 `body` 的真分支上利用 $i < n$ 。`main` 还建立了 $0 \leq n \leq 8$ ，但这个事实要穿过函数边界，需要内联、过程间分析或显式参数契约。于是，同一套方程也揭示了工程权衡：更强的关系域和过程间上下文能证明更多边界与别名事实，却消耗更多时间和内存；生产编译器必须在精度、可扩展性和诊断稳定性之间选择。

SSA 让这类分析变得稀疏：`%acc.next` 的使用者可由 `def-use` 链直接找到，无须在每个程序点维护所有变量。但 `load` 的结果仍可能受别名 `store` 或未知调用影响，所以普通 SSA 从未消灭内存分析，只是把纯值问题从内存问题中剥离出来。

5.2 合法改写是语义精化，不是文本相似

设局部程序片段 p 在前置条件 C 下被改写为 q ，并把两者的执行投影到共同的可观察行为集合 Beh 。有未定义行为或未指定选择时，要求行为完全相等往往过强；更合适的局部义务是

$$C \implies \text{Beh}(q) \subseteq \text{Beh}(p). \quad (8)$$

即目标可以选定源程序允许的某种实现，但不能产生源程序没有承诺的行为。这里的包含方向称为语义精化 (semantic refinement)；它不是在声称本文已经形式化整个 LLVM，而是给“合法”一个可讨论的方向 [50,67]。

案例的截断函数在 $lo \leq hi$ 时满足

$$c(x) = \begin{cases} lo, & x < lo, \\ hi, & x \geq lo \wedge x > hi, \\ x, & \text{otherwise} \end{cases} \\ = \max(lo, \min(x, hi)).$$

因此，控制流形式可以合法地规范化为比较、`select` 与 `llvm.smin` 组成的无分支值表达式。它保留的是截断函数，而不是原来“经过了哪些块”的轨迹；若边界关系未知，或分支中含有不能无条件执行的副作用/陷阱，把控制依赖直接变成数据依赖就未必合法。

溢出与 `poison` 也属于同一道合法性门。把普通 `add` 换成 `add nsw` 会把原本定义的环绕结果变成 `poison`，只有在源语义已排除有符号环绕时才成立；对可能为 `poison` 的值进行投机执行，还必须保证它不会比原程序更早流入分支、地址或其他触发未定义行为的位置。`freeze` 有时能固定这种不确定性，但会改变可用的等价式，不应被当成万能修补。换言之，代数恒等式、LLVM 位向量语义和源语言溢出规则是三个必须对齐的层次。

5.3 从可寻址变量到可向量化归约

优化的关键通常不是一个神奇 `pass`，而是表示形状逐步消除偶然约束。未优化的前端 IR 常把 `i`、`acc` 和临时量放入 `alloca`：

```
%acc = alloca i32, align 4
store i32 0, ptr %acc, align 4
...
%old = load i32, ptr %acc, align 4
%new = add i32 %old, %term
store i32 %new, ptr %acc, align 4
```

当地址没有逸出且访问可提升时，`mem2reg/SROA` 将它改成循环头 `phi` 与直接的 `def-use` 链；控制流简化和指令组合再把截断的块结构规范化为逐元素的值运算 [46]。这些改写本身未必带来明显速度提升，却使“归纳变量”“归约”和“逐项 `clamp`”成为下游能识别的形状。

随后，循环向量化器可以构造宽向量主体、水平归约与标量尾部。对本例而言，每个迭代读取 `a[i]` 和 `b[i]`、逐项相乘并截断，然后加到累加器。向量化要保持以下依赖：同一元素内先乘再截断，所有截断结果最终参与归约；它可以改变独立元素之间的执行次序。由于每项已在 $[-8, 12]$ 内、最多八项，整数和位于

Table 2. 案例优化链中的形状、机会与约束

表示变化	为下一步显露的结构	主要约束
load/store 局部量 → SSA	归纳变量与累加递推	地址逸出、volatile、未知内存效应
分支截断 → 值表达式	逐通道的 min/max/select	副作用、陷阱、poison 与边界关系
标量迭代 → 向量主体	独立元素并行与水平归约	循环携带依赖、别名、溢出/浮点语义
主体加标量尾部	任意 n 的覆盖	额外 CFG、代码尺寸与短循环开销

[−64, 96]，重排归约不会产生有符号溢出。若把元素换成通常的 IEEE-754 浮点数，加法结合律会受舍入破坏；没有 fast-math 一类明确放宽时，同样的重排不再自动合法。

内存是另一道约束。本例只读取 a 与 b ，即使两者别名，同时读取相同位置也不制造跨迭代读写依赖，所以不必为了这次向量化虚构 `noalias`。若循环改为写 $a[i]$ 并读取 $b[i-1]$ ，重叠实参可能让下一迭代读到刚写入的数据；优化器必须证明不别名、生成运行时别名检查并保留标量版本，或放弃向量化 [39,40]。更激进的属性不是“性能开关”，而是调用者必须共同遵守的语义承诺。

表 2 把这条链压缩为机制而非 pass 清单。每一步既消除一种表示偶然性，也可能在新的约束前停下。

5.4 盈利性：正确候选仍可能被拒绝

通过合法性门后，优化器面对的不是唯一“最优 IR”，而是一组语义允许的候选。可把目标相关选择概括为

$$Q^* \in \arg \min_{Q \in \mathcal{L}(P)} (C_{time}(Q), C_{size}(Q), C_{compile}(Q), C_{energy}(Q)).$$

其中 $\mathcal{L}(P)$ 是通过合法性检查的候选集合。这是一个多目标问题而非可直接比较的四元组；实际成本模型依优化级别、目标变换信息 (Target Transform Info, TTI)、代码布局和经验启发式，把目标折算或分阶段决策。

向量化清楚展示了这种区别。面向具有合适 SIMD 指令的 x86-64，宽 load、逐通道截断和归约可能摊薄循环控制成本；面向当前案例的 RV64GC（未启用 RISC-V V 扩展），同样的向量 IR 可能需要拆回标量，代码尺寸和尾部处理反而更贵。因此保存的 RV64GC 版本采用标量 `lw/mulw/addw` 循环并不与“向量化合法”矛盾：合法性回答能否，目标成本回答是否值得。短循环、冷代码或寄存器压力较高时，即便目标有向量指令，也可能合理拒绝。

盈利性判断天然不完美。静态模型不知道部署时输入分布和微架构状态；基于剖面的优化能补充频率信息，却会带来训练代表性与构建流程成本。更全面的全局搜索也许找到更好的代码形状，却增加编译延迟。LLVM 的 Pass Manager 因而把分析结果按 IR 单元缓存，并要求变换准确声明哪些结果仍有效；失效过多会重复计算，失效不足则会让后续优化依赖过期事实 [49]。

本层原则。优化可以看成“抽象解释建立事实，语义精化筛掉错误候选，成本模型在合法候选中作出目标相关选择”。这比背诵 pass 名单更接近真实系统：规范化负责让机会可见，分析负责证明边界，盈利性负责接受现实资源约束。选定的 IR 仍只是一个抽象机器程序；下一层将把这些自由度兑换成 RV64 的指令、寄存器和调用约定。

6 降到 RV64：约束求解、机器资源与二进制契约

LLVM IR 故意把机器资源藏在抽象之后：SSA 名称仿佛无限，整数位宽由类型给出，phi 直接表达控制流汇合，调用只写函数类型。RV64 处理器却只有有限寄存器、受编码约束的指令，以及由平台规定的函数接口。后端（backend）因此不是逐行翻译器，而是一个连续的约束求解过程：它要选择目标能够执行的操作，为满足数据依赖的操作排定顺序，把同时存活的值放进有限寄存器，最后兑现应用二进制接口（application binary interface, ABI）。LLVM 的典型主干是

LLVM IR $\rightarrow$ legalization/selection $\rightarrow$ MachineInstr $\rightarrow$ scheduling $\rightarrow$
register allocation $\rightarrow$ frame lowering $\rightarrow$ MC

其中组合、调度和机器级清理可能出现多次；这是一组相互制约的问题，而不是不可变的 pass 清单 [45]。本节沿 `clipped_dot` 走完这条链：既给出足以解释设计的数学模型，也随即指出生产编译器为何不能照搬教科书中的理想算法。图 6 先把同一组操作在四种约束视图中并排，后文再逐一展开。

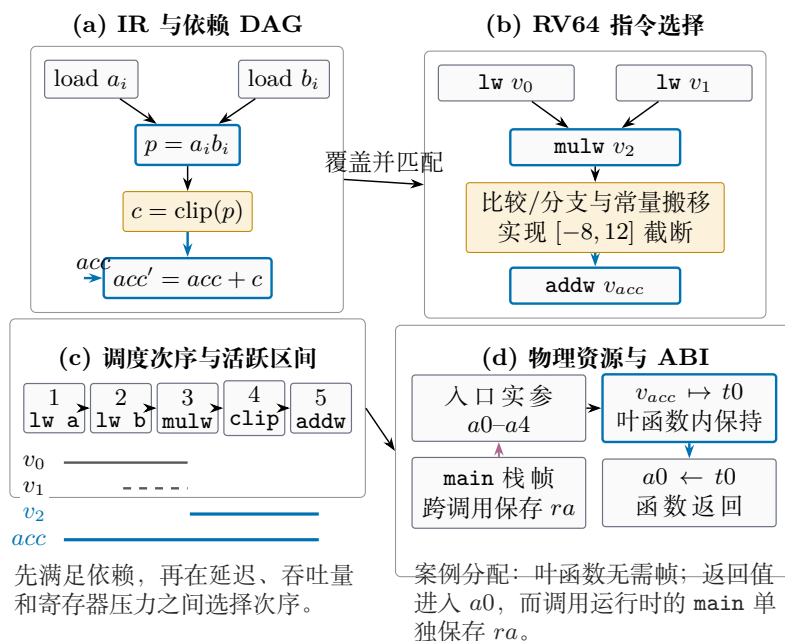


Fig. 6. 后端不是四个彼此独立的阶段：同一组载入、乘法、截断与累加先受数据依赖约束，再被目标指令覆盖；调度改变活跃区间，寄存器分配与 ABI 又把局部选择变成跨调用必须履行的契约。

6.1 指令选择：从语义关系到有成本的覆盖

设源操作为 u ，目标 T 上可实现它的机器序列集合为 $L_T(u)$ 。最朴素的选择问题是

$$m^* = \arg \min_{m \in L_T(u)} C_T(m) \quad \text{s.t.} \quad \text{Beh}_T(m) \subseteq \text{Beh}_{\text{IR}}(u). \quad (9)$$

行为集合的包含方向沿用式 (1)：目标序列只能选择 IR 在既定未定义行为、内存模型和调用约定下允许的行为。成本 C_T 可以综合指令数、延迟、吞吐、代码尺寸和寄存器压力。式 9 揭示了两个重点：选择的单位未必是一条 IR 指令；局部最短的序列也未必给出全局最优代码。

目标不支持的操作或类型先经过合法化 (legalization)。例如，某目标可把过宽整数拆成若干机器字，也可把罕见运算改写为运行库调用。对改写 $u \mapsto (u_1, \dots, u_k)$ ，合法化承担的基本义务是

$$\text{Sem}(u) = \text{Sem}(u_1; \dots; u_k), \quad u_i \in \mathcal{O}_T,$$

其中 $\mathcal{O}_T$ 是目标可继续处理的操作集合。现实中“可处理”还受寄存器类、立即数范围、寻址模式和向量形状约束；因此合法化、组合与选择必须反馈迭代，而不能被隔成三张互不相干的查找表。

LLVM 长期使用 SelectionDAG：把基本块附近的操作及依赖建成有向无环图 (directed acyclic graph, DAG)，在其中做组合、类型与操作合法化、模式匹配和调度。GlobalISel 则显式经过 IRTranslator、通用机器 IR (generic Machine IR, gMIR)、Legalizer、RegisterBankSelect 和 InstructionSelect；它便于在更大范围保留机器无关信息，但各目标的覆盖程度和回退策略并不相同 [42,43]。二者都说明了同一个工程事实：式 9 是设计准绳，而工业实现依赖规则生成器、目标钩子和启发式搜索来控制编译时间。

在案例中。i32 乘法与加法落到 `mulw/addw`，数组寻址落到移位、整数加法和 `lw`。而 IR 的比较加条件分支可以合并为直接比较寄存器的 `blt`；机器没有必要先物化一个布尔值。源程序写下标 i ，后端也可以把循环改成两个递增指针和一个末端指针：选择的是等价的计算形状，而不是相似的表面语法。

6.2 RV64 的宽度纪律：地址为 64 位，值仍可为 32 位

实验目标 RV64GC/LP64D 同时包含三层契约。指令集体系结构 (instruction set architecture, ISA) 规定可用的机器操作；LP64D ABI 规定类型布局、传参和栈；目标三元组还选择操作系统与对象格式。RV64 的 $\text{XLEN} = 64$ 不会把 SysY/C 的 `int` 自动变成 64 位。对寄存器值 x ，RV64 的字操作可以概括为

$$\text{lw}(p) = \text{sext}_{32 \rightarrow 64}(M[p, p+4]), \quad (10)$$

$$\text{addw}(x, y) = \text{sext}_{32 \rightarrow 64}((x + y) \bmod 2^{32}), \quad (11)$$

$$\text{mulw}(x, y) = \text{sext}_{32 \rightarrow 64}((xy) \bmod 2^{32}). \quad (12)$$

因此地址计算使用 64 位 `add`，数组元素与累加器使用 `lw/mulw/addw`[59]。若把 `lw` 换成 `ld`，一次会读入两个元素；若无条件把 W 类算术换成 64 位算术，32 位回绕后的比较结果可能改变。目标宽度不是性能装饰，而是源类型语义的一部分。

RISC-V 的 G 是 I、M、A、F、D、Zicsr、Zifencei 的约定集合，C 扩展另行提供 16 位压缩编码 [60]。案例的整数内核只需要 I 与 M；目标允许 A/F/D 并不意味着每个函数都使用原子或浮点指令。模块化 ISA 给硬件实现留出选择空间，但软件分发必须把实际 ISA 与 ABI 基线说清，否则同名“RISC-V 程序”仍可能无法互操作。

6.3 调度：偏序的线性扩展，而非任意重排

选择之后得到的依赖 DAG 记为 $G = (V, E)$ 。一份调度是位置函数 $s : V \rightarrow \mathbb{N}$ ；若边 (u, v) 的延迟为 $\ell(u, v)$ ，则必须满足

$$s(v) \geq s(u) + \ell(u, v), \quad (13)$$

并且每周期占用某类执行资源的操作数不超过该资源容量。若只最小化尾部时间 $\max_v(s(v) + \ell_v)$ ，调度器会积极拉开独立操作；但这常使更多中间值同时存活。一个更诚实的目标是

$$\min_s (\alpha L(s) + \beta P(s) + \gamma S(s)), \quad (14)$$

其中 L 是关键路径或估计周期， P 是峰值寄存器压力， S 是代码尺寸。系数由目标微架构和优化级别决定。

案例每轮的两个 `lw` 相互独立，可在 `mulw` 之前并行准备；乘积必须等两次读取完成，累加又依赖截断后的乘积。提前读取下一轮可能隐藏访存延迟，却会增加跨迭代活跃值。LLVM 因而使用目标调度模型和启发式列表调度，而不是精确求解式 14：通用最优调度代价过高，且真实性能还受缓存、分支预测与具体核实现影响。生产后端追求的是在编译时间预算内稳定地产生好代码，而不是为抽象机器求得不可迁移的数学最优。

6.4 活跃性：用不动点回答“这个值还要不要”

寄存器分配首先需要知道值在何处仍会被使用。对控制流图中的基本块 b ，记 use_b 为先于本块定义而使用的值， def_b 为本块定义，经典活跃性 (liveness) 方程为

$$\text{out}_b = \bigcup_{s \in \text{succ}(b)} \text{in}_s, \quad (15)$$

$$\text{in}_b = \text{use}_b \cup (\text{out}_b - \text{def}_b). \quad (16)$$

从空集反复应用式 16 会在有限值域上到达最小不动点。它不是抽象仪式：若输入长度原在 `a2`，计算完末端地址后该长度不再被使用，`a2` 就可承载数组元素和乘积。无限个 SSA 名称由此被压缩到同一物理寄存器的不相交生命周期中。

把活跃区间相交的值连边，得到干涉图 $H = (V_H, E_H)$ 。若某寄存器类有 K 个可用寄存器，理想化分配就是寻找着色

$$c : V_H \rightarrow \{1, \dots, K\}, \quad (u, v) \in E_H \Rightarrow c(u) \neq c(v).$$

图着色说明了资源冲突的本质 [5]，却不足以描述生产分配器：物理寄存器可能互相别名，调用参数已经预着色，不同指令限制不同寄存器类，活跃区间还能拆分。LLVM 的分配器综合 live interval、驱逐、合并、拆分和 spill 成本 [45]。当 K 不够时，值可溢出 (spill) 到栈，也可对廉价常量做重物化 (rematerialization)；后者用计算换访存。案例内核压力很低， $t0--t6$ 与参数寄存器已经足够，所以无需为中间值建立栈槽。

调度与分配在这里真正相遇：拉长独立 load 与消费点之间的距离，可能降低流水线停顿，却会在干涉图中增加边。实际后端常在分配前按压力调度，并在分配后修补 hazard；不存在适合所有目标的“先完美调度、再独立着色”。

6.5 ABI：把函数调用写成状态变换契约

把机器状态写成 $\sigma = (R, M, pc)$ 。对函数 f ，ABI 不规定内部路径，只要求从满足入口条件的 σ_{in} 到返回状态 σ_{out} 时：参数与返回值位于约定位置；被调用者保存寄存器保持不变；栈指针恢复；内存副作用符合语言与链接契约。对 RV64 的标准整数调用约定， $a0--a7$ 传递参数， $a0--a1$ 也承载返回值； $t0--t6$ 与 $a0--a7$ 为调用者保存， $s0--s11$ 为被调用者保存， ra 保存返回地址 [56]。在每个调用边界还要求

$$sp \equiv 0 \pmod{16}. \quad (17)$$

`clipped_dot(a,b,n,lo,hi)` 的五个参数自然进入 $a0--a4$ ，结果回到 $a0$ 。它不调用其他函数，也不使用必须恢复的 s 寄存器，因而可以既不调整 sp ，也不保存 ra 。这不是省略了“标准序言”，而是 ABI 从未要求固定模板。相反，`main` 会调用 I/O 与内核；每次 `call` 都改写 ra ，故它在帧中保存自己的返回地址。这里的 `sd/ld ra` 是调用者为跨调用保存架构状态，不应与寄存器不足造成的 spill 混为一谈。

案例的 80-byte 栈帧同时容纳两个 8 元素 i32 数组和返回地址，并满足式 17。帧降低 (frame lowering) 在寄存器分配之后才掌握 spill 槽、局部对象、被保存寄存器和对齐的总需求，由此把抽象 frame index 变成 sp 相对偏移。优化器若把数组常量移入只读区，帧就可能大幅缩小；栈帧尺寸取决于最终存储策略，而不是源代码中局部变量的计数。

6.6 读一遍手写 RV64：让每条指令承担一种责任

下面保留案例核心的控制结构，并在右侧标出语义责任：

```
li      t0, 0           # acc = 0
li      t1, 0           # i = 0
.Lloop:
bge     t1, a2, .Ldone   # while (i < n)
slli    t2, t1, 2        # byte offset = 4*i
add     t3, a0, t2       # &a[i], 64-bit address
lw      t4, 0(t3)        # signed i32 a[i]
add     t3, a1, t2       # &b[i]
lw      t5, 0(t3)        # signed i32 b[i]
mulw    t4, t4, t5       # i32 product
blt     t4, a3, .Llower  # product < lo
blt     a4, t4, .Lupper  # hi < product
.Laccumulate:
addw    t0, t0, t4       # acc += clipped product
addiw   t1, t1, 1        # ++i in the source value domain
j       .Lloop
```

```
.Llower: mv t4, a3; j .Laccumulate
.Lupper: mv t4, a4; j .Laccumulate
.Ldone:  mv a0, t0; ret
```

控制流把三种截断结果汇入唯一累加块，因而循环回边和状态更新只有一份。用函数

$$\text{clip}_{lo,hi}(x) = \begin{cases} lo, & x < lo, \\ x, & lo \leq x \leq hi, \\ hi, & x > hi \end{cases}$$

来看，两条条件分支只是该分段函数在没有条件码寄存器的 ISA 上的一种实现。工程上还可尝试条件选择或无分支序列；它们是否更快取决于分支可预测性、指令条数与关键路径，不能仅凭“无分支”标签判断。

输入 $n = 8$ 时，乘积为 $(-8, -3, 7, 12, -9, -10, 12, 15)$ ，截断后为 $(-8, -3, 7, 12, -8, -8, 12, 12)$ ，和为 16。这组数值在机器层最有解释力之处，不是充当测试用例，而是把 `mulw`、两个有符号 `blt` 和 `addw` 串成一条可心算的语义链。

`main` 的关键接口则是：

```
addi    sp, sp, -80
sd      ra, 72(sp)          # preserve our caller's return address
...
call    getint              # two i32[8] arrays at sp and sp+32
...
mv      a0, sp              # clamp n to [0,8] in a2
addi    a1, sp, 32
li      a3, -8
li      a4, 12
call    clipped_dot
call    putint              # return in a0 is already argument 0
...
ld      ra, 72(sp)
addi    sp, sp, 80
ret
```

返回值与下一个调用的首参数共用 `a0`，所以不需要搬移。这种“接口对齐”既来自 ABI，也来自寄存器合并；它展示了稳定约定如何给局部优化创造机会。

6.7 伪指令、汇编指示与 MC：文字还不是机器字节

`li`、`mv`、`j`、`ret` 和 `call` 是伪指令 (pseudoinstruction)。`mv` 常可编码为加零，`ret` 通常归约为经 `ra` 的 `jalr`；`li` 可能需要一条或多条真实指令。`call symbol` 保留尚未知的符号距离，由汇编器产生 PC 相对指令与重定位，链接器还可在距离允许时松弛为更短形式 [58,57]。因此“一行汇编等于一条机器指令”并不是接口保证。

同一文件中还有汇编指示 (assembler directive)，它们不由 CPU 执行。`.text` 选择节，`.globl`、`.type` 与 `.size` 描述符号；`.attribute` 和 `.note.GNU-stack` 则记录目标属性。代码与元数据必须一起进入对象文件，因为独立编译要求链接器能够在没有源程序的情况下解析符号和重定位。

LLVM 在最后把 `MachineInstr` 降为 MC 层的 `MCInst`。`MCAsmStreamer` 可打印 `.s`，`MCObjectStreamer` 与编码器则可直接形成对象文件；二者共享的核心是指令、标签、节和 `fixup`，而不是某份汇编文本 [45]。这也解释了后端与下一

章的边界：后端已经确定“执行什么”和“遵守何种机器接口”，汇编器与链接器继续决定这些选择如何编码、定位并组合成进程可装载的映像。

6.8 从模型中抽取工程原则

这一层的形式化并不要求把 LLVM 描述成一个求全局最优解的数学机器。恰恰相反，模型的价值在于标出不可违反的约束和可以折衷的目标：

1. 语义细化与 ABI 是硬约束；指令条数、延迟和代码尺寸是成本，不能为了成本破坏契约。
2. 合法化、选择、调度和分配共享信息。把它们完全割裂会制造局部最优，例如为隐藏延迟而造成无法承受的寄存器压力。
3. 教科书模型给出问题骨架，不等于生产算法。DAG 覆盖、最优调度和图着色都可能计算昂贵；LLVM 用目标描述、启发式和分阶段修正换取可预测的编译时间。
4. ISA 提供机制，ABI 提供社会契约。前者可以模块化演进，后者必须稳定，独立编译的软件生态才能长期组合。

于是，“降到机器”不是逐步丢失高级信息，而是把高级语义逐层兑现为更具体的约束：SSA 数据依赖成为调度偏序，值的未来使用成为活跃性不动点，有限寄存器成为干涉关系，函数类型成为 ABI 状态变换，符号调用成为等待链接器求值的重定位。抽象没有消失；它换成了处理器和工具链能够共同执行的形式。

7 从符号调用到进程映像：汇编、链接与装载

上一节停在一条看似平常的指令：`clipped_dot` 的返回值已经位于 `a0`，接下来执行 `call putint`。然而汇编器尚不知道 `putint` 最终位于哪个地址，静态库尚不知道自己是否会被采用，内核也不关心 `putint` 这个名字。这个调用因而是观察编译系统后半程的理想切片：同一意图依次表现为符号引用、重定位约束、归档需求、虚拟地址，最后才成为一次控制转移。这里的“未知”并非信息遗漏，而是被有意推迟的符号义务 (symbolic obligation)；每一层只解决自己已有信息足以决定的问题。

7.1 汇编器：把未定地址编码成代数问题

将一个可重定位目标文件抽象为三元组

$$O = (B, \Sigma, R).$$

其中 B 是按节组织的字节， Σ 是从符号名到“定义位置、绑定、类型或未定义”的映射， R 是尚待求值的重定位集合。汇编器能确定局部标签在某个输入节中的偏移，因此可以填入大部分 B ；对于跨对象调用，它并不猜测地址，而是在 Σ 中留下未定义符号，并在 R 中记录“何处、按哪种规则、相对于哪个符号”完成计算。经典的两遍汇编 (two-pass assembly) 恰好揭示这一依赖：先建立局部布局，再编码引用；真实实现可以合并扫描、迭代布局或采用片段数据结构，而不改变这种信息顺序。

在 RISC-V 中，`call putint` 是伪指令 (pseudoinstruction)。一般距离下，它可展开为

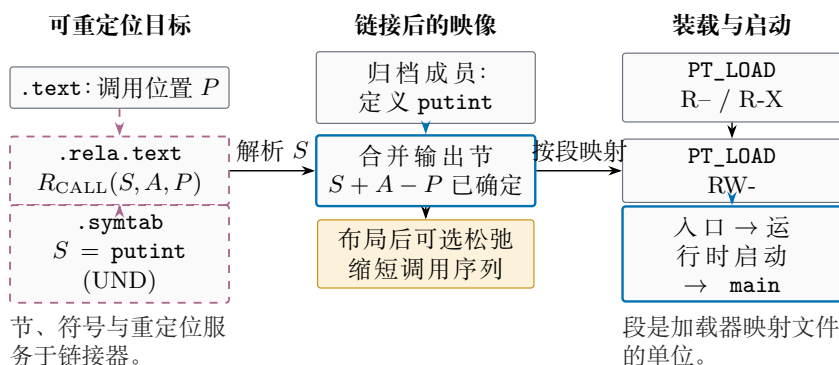


Fig. 7. 对 `putint` 的调用从“带类型的未决引用”变为链接后的地址关系，再随可装载段进入进程映像；节、符号、重定位和段分别服务于不同的决定。

```
auipc ra, 0          # R_RISCV_CALL_PLT putint
jalr  ra, 0(ra)      # 与上一条共同形成 PC 相对调用
```

零立即数不是错误地址；指令字节与 `R_RISCV_CALL_PLT` 合在一起才是完整表达。若 S 是 `putint` 的最终地址， A 是显式加数， P 是应用重定位的位置，则链接器最终求值

$$\Delta = S + A - P. \quad (18)$$

RISC-V 再把 Δ 分配给高、低立即数字段。忽略具体编码位的排列，可写成

$$h = \left\lfloor \frac{\Delta + 2^{11}}{2^{12}} \right\rfloor, \quad \ell = \Delta - 2^{12}h,$$

使 $\ell \in [-2^{11}, 2^{11} - 1]$ 能由有符号 12 位立即数表示。加上 2^{11} 的舍入不是技巧，而是低 12 位可能为负所要求的补偿；这也说明重定位不能简化成“把地址填进一个洞”。具体类型还规定取值范围、指令位域以及溢出时应当报错的条件 [56]。

这种延期同时给优化留下空间。若最终的 Δ 落入 `jal` 的范围，且对象以 `R_RISCV_RELAX` 授权，链接器可把 `auipc+jalr` 收缩为一条跳转；若距离较远，就保留一般形式 [57]。工程权衡很清楚：汇编器采用稍长但位置无关的保守编码，避免过早绑定；链接器掌握全局布局后再消除不必要的通用性。若手工写死偏移，代码既不能随布局移动，也失去松弛机会。

7.2 ELF 对象：链接视图与装载视图不是一回事

可重定位 ELF (Executable and Linkable Format) 对象 `ET_REL` 主要呈现节 (section)：`.text` 保存待布置的指令字节，`.symtab/.strtab` 保存符号及名字，`.rela.text` 保存带显式加数的重定位。符号值此时通常只是定义节内偏移；`putint` 的 `SHN_UNDEF` 则表示“由链接环境提供定义”。因此，可重定位对象不是残缺的可执行文件，而是一份边界清晰的链接语言：它提交已经确定的局部事实，同时把式 (18) 留给拥有全局地址的人求值 [66]。

ELF 的另一个视图是段 (segment)，由程序头描述内核应映射的文件区间、虚拟地址和 R/W/X 权限。节回答“这些字节在链接时扮演什么角色”，段回答“这些页在运行时如何进入地址空间”。一个 `PT_LOAD` 可以覆盖多个节；调试节和完整静态符号表可以留在文件中却不被映射，反之，一个没有程序头的普通 `ET_REL` 对象仍然完全合法。把“加载 ELF”说成“依次加载 `.text`、`.data`、`.bss`”虽便于入门，却混淆了两套索引。尤其是 `.bss`：链接视图通常以 `SHT_NOBITS` 表示它不占文件字节；装载视图则由 $p_{memsz} > p_{filesz}$ 的可写段表达尾部零填充。

7.3 链接器：求符号环境的不动点

链接器首先为名字选择定义。令 $D(X)$ 是已选择对象集合 X 提供的全局定义， $U(X)$ 是其中尚未满足的强引用。普通对象总被纳入；静态归档 (archive) 则是带符号索引的对象集合，只在某成员能满足当前需求时才抽取。其核心过程可写成最小不动点：

$$\begin{aligned} X_0 &= \{\text{命令行中显式对象}\}, \\ X_{k+1} &= X_k \cup \{m \mid m \text{ 是当前归档成员且 } D(\{m\}) \cap U(X_k) \neq \emptyset\}. \end{aligned}$$

当 $X_{k+1} = X_k$ 时，本轮归档搜索稳定。于是，手写对象先提出 `putint` 需求，`libsysy` 的符号索引再把需求导向定义该函数的成员；若归档出现在提出需求之前，传统的从左到右扫描通常不会自动回头 [10,11]。--start-group 通过反复扫描处理归档间循环依赖，但会增加链接成本；--whole-archive 强制纳入所有成员，又可能放大代码和初始化副作用。正确的对象次序通常比这两种重锤更简单。

符号解析并非“同名即兼容”。课程随附的 RISC-V 归档由 newlib 环境构建，内部还依赖 `_impure_ptr`；本文的 GNU/Linux 交叉环境使用 glibc，因而从同一份 `sylib.c` 重建运行时。这个插曲只说明一个重要边界：ISA 决定机器指令，处理器 ABI (psABI) 决定寄存器与数据布局，系统 C 库及启动环境又形成更外层契约。文件同为 ELF64 RISC-V，仅证明容器和机器类别相容，不能推出内部运行时符号也相容。比伪造缺失符号更可靠的工程选择，是让库实现与最终 sysroot 共享同一构建环境。

定义选定后，链接脚本把输入节布置到输出节，地址布局赋予式 (18) 中的 S 与 P ，链接器继而应用重定位、执行目标相关松弛，并生成程序头 [12]。这几步存在反馈：松弛会缩短代码，改变后继地址，布局与重定位因而可能迭代至稳定。抽象地说，链接器寻找一个布局 L ，使每个重定位约束在 L 下可编码，并使输出段满足对齐和权限合同。这个视角比背诵固定步骤更准确：成熟链接器可以交错扫描、布局和松弛，但不能在知道地址之前正确求出 PC 相对位移。

7.4 加载器：名字消失之后，地址成为一次执行

最终静态可执行文件通常是 `ET_EXEC`，位置无关可执行文件通常以 `ET_DYN` 表示。内核执行 `execve` 时读取程序头，将每个 `PT_LOAD` 的文件区间映射到虚拟地址空间，并对 $p_{memsz} - p_{filesz}$ 的尾部清零；一个基本一致性条件是

$$p_{vaddr} \equiv p_{offset} \pmod{p_{align}}, \quad 0 \leq p_{filesz} \leq p_{memsz}.$$

Table 3. 案例的端到端贯通结果。

维度	结果
可执行形式	C、SysY、手写 IR、手写 RV64
输入到输出	$(-1, 0, 1, 4, 8, 10) \mapsto (0, 0, -8, 8, 16, 16)$
退出状态	全部执行均为 0

动态程序还通过 `PT_INTERP` 把共享对象装载和动态重定位委托给用户态动态链接器 [65]。到这个阶段，内核只看地址、页与权限；源语言函数名是否保留在非装载符号表中，不影响 CPU 跳转。

ELF 入口通常是 `_start`，而非 `main`。编译器驱动加入的 C 运行时启动代码（C runtime startup, CRT）接收内核建立的参数、环境和辅助向量，初始化 `libc`，运行构造器，调用 `main`，再把返回值交给终止流程并运行析构器 [19]。因此，本案例中 `putint` 的完整旅程是：汇编源提出一个名字；目标文件把它保存为重定位义务；链接器从归档中选择定义并求出地址；内核与 CRT 建立执行上下文；CPU 最终按已修补的调用序列进入运行时实现。所谓“编译成机器码”若省略这条链，便漏掉了让机器码真正成为程序的制度部分。

7.5 一次小规模贯通检查

为确认上述工具链确实能承载案例，而不是把测试本身写成章节主题，我们只保留一个紧凑的端到端检查。C 观察载体、SysY 前端路径、手写 LLVM IR 和手写 RV64 汇编分别形成可执行文件；对同一组边界输入，它们得到相同的标准输出和退出状态。该结果说明本章追踪的四条路径能够贯通当前环境，不承担“从有限样例证明全体程序等价”的任务。

运行时析构器还向标准错误写出计时汇总；它属于库的生命周期行为，而非点积函数的返回值。将标准输出、标准错误与退出码分开，足以避免把运行时诊断误当成算法结果。更一般地，测试适合发现实现没有遵守已经说明的合同，形式化语义则说明合同是什么：二者互补，但不应让可复现性清单取代对汇编、重定位与装载机制本身的解释。

从这一条 `putint` 调用可以提炼出比工具命令更持久的原则：跨层设计的关键，不是尽早消灭未知量，而是把未知量变成下一层可求解、可检查且不丢语义的义务。汇编器保留符号与重定位，链接器保留装载所需的段和动态元数据，加载器再把相对结构实例化为一次具体进程。延迟决定不是拖延工作，而是让决定发生在信息最充分的层次；这与前文从源语言逐步降低到指令的思想正好首尾相接。

8 多层中间表示：让信息在需要时才消失

LLVM IR 适合表达已经具体化的控制流、内存访问和调用约定，但编译器并不总想立刻回答这些问题。对截断点积而言，式 (2) 首先说的是三个结构：逐元素相乘、逐元素限幅、沿索引维归约。若前端立即把它们展开为指针递增和条件跳转，后端仍可能生成好代码，却必须从偶然的循环形状中重新辨认这些结构。多层中间表示（multi-level intermediate representation）的核心思想不是增

加更多“必经阶段”，而是延长有用信息的寿命，并在决策真正发生时才将其具体化。MLIR 为这种做法提供了可扩展的操作、类型、属性、接口和转换基础设施 [22,34]。

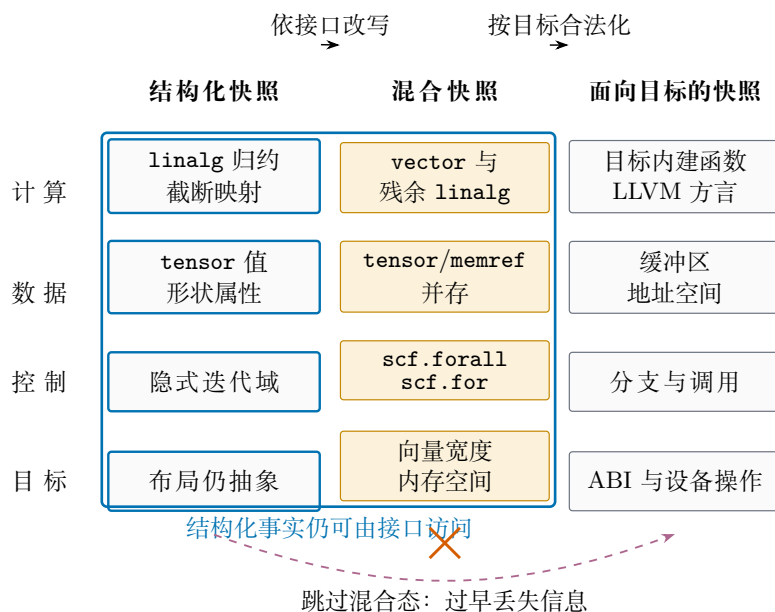


Fig. 8. 渐进式降低改变的是模块中各种抽象的组成比例：计算、数据、控制和目标约束可以按不同节奏显式化；接口与合法性条件使混合表示仍可组合。

8.1 混合方言是正常状态

方言 (dialect) 是语义命名空间，不是流水线中的独占阶段。一个合法模块可以同时使用 `linalg` 描述结构化归约、`tensor` 表达值语义、`arith` 表达标量比较、`scf` 表达循环，并夹杂尚未转换的调用。图 8 因而不是一列互斥方框：降低过程改变的是模块中各种抽象的比例，而不是按下开关后把整个程序瞬间换成“下一种 IR”。

对长度 k 的截断点积，可把核心计算写成

$$p_i = a_i b_i, \quad c_i = \min(12, \max(-8, p_i)), \quad r = \sum_{i=0}^{k-1} c_i. \quad (19)$$

前两个式子是同一索引空间上的结构化映射，最后一个式子是以 0 为初值的加法规约。这一描述保存了索引映射、归约维和逐元素区域，因此融合、分块和向量化不必先做循环识别。边界 $0 \leq k \leq 8$ 与 $-8 \leq c_i \leq 12$ 又给出 $-64 \leq r \leq 96$,

所以这里把归约重排为加法树不会引入有符号溢出；若删除长度上界，这项工程自由便随之消失。形式化不变量在此不是装饰，而是优化许可。

仅有许多方言并不足以形成系统。接口 (interface) 让通用变换询问一个操作“拥有什么能力”，而不是枚举它“属于哪个名字”。例如，结构化操作接口暴露迭代器和索引映射，目的地风格接口显式给出输出或初值，内存效果接口说明读写行为，缓冲化接口描述别名和存储改写 [33,37]。因此，新的领域操作可以接入已有的分块或缓冲化算法；反过来，若一个操作没有声明相应契约，通用算法就不应仅凭外形猜测其语义。这是可扩展性与可推理性之间最重要的平衡。

8.2 降低是一个合法性问题

模式改写 (pattern rewriting) 给出局部等价替换；方言转换 (dialect conversion) 进一步规定整个转换结束时允许留下什么。令模块中的操作集合为 O ，转换目标给出合法性谓词 (legality predicate)

$$L_t(o, \Gamma) \in \{\text{true}, \text{false}\}, \quad \text{Legal}_t(M) \iff \forall o \in O(M), L_t(o, \Gamma_M), \quad (20)$$

其中 t 是本次转换目标， Γ_M 包含类型、属性和所在区域等上下文。静态合法性只依赖操作种类；动态合法性还可要求函数签名已经转换、布局满足限制或某个属性已经确定。部分转换只要求被声明为非法的操作消失，完全转换则要求范围内每个操作都被目标接纳 [32]。于是“中间模块同时含有 `linalg`、`scf` 和 `vector`”并不表示流水线只做了一半；只要式 (20) 对当前目标成立，它就是刻意选择的稳定交接面。

这种设计把一个难以维护的全局转换拆成可组合的责任：规则只需把自己承诺处理的非法操作改写为更接近目标的形式，框架负责反复应用规则并检查终态。代价也很现实。合法集合定义得过宽，错误会被推迟到更难诊断的阶段；定义得过窄，则每个实验性操作都必须立刻获得完整后端支持。好的转换边界不是“越低越严谨”，而是在当前决策所需的信息与下一阶段可接受的复杂度之间取平衡。

8.3 类型转换需要跨边界的物化

操作合法化不能脱离类型转换 (type conversion)。TypeConverter 可以把一个源类型映射为零个、一个或多个目标类型；例如，排序 `memref` 在 LLVM ABI 边界常展开为分配指针、对齐指针、偏移、各维大小与步长，而不是简单改名为一个 `ptr` [32,29]。若生产者已转换而消费者仍停留在源方言，两者的 SSA 类型便不再相同。源物化、目标物化与参数物化 (materialization) 在这个断面构造桥接值，使局部转换仍能保持整个使用-定义链类型正确。

临时的 `builtin.unrealized_conversion_cast` 可用于推迟桥接细节，但它没有独立的机器语义。允许它永久流入代码生成，相当于把尚未解决的类型义务伪装成普通操作。工程上应把这类 `cast` 看作“语义债务”：混合 IR 内可以短暂存在，越过最终翻译或 ABI 边界前必须被真正的物化规则消除。这也说明渐进式降低并非放弃严格性；它只是把严格性表述为每个边界各自可检查的合同。

8.4 缓冲化改变了程序讨论的问题

张量值回答“这个计算产生什么值”，缓冲区还要回答“该值存在哪里、与谁别名、由谁释放”。缓冲化 (bufferization) 因此不是换一种类型拼写，而是从值等价进入存储身份。One-Shot Bufferize 沿 SSA 使用一定义链分析读写冲突，尽量将目的地风格操作原地化；当原地更新可能改变仍可观察的旧值时，就必须分配或复制 [31]。

在式 (19) 中， p_i 与 c_i 没有必要物化为完整数组：映射可以同归约融合，每次生成一个 c_i 后立即累加。保持张量和结构化操作直到完成融合，使这个结论直接可见。若过早缓冲化，中间数组的分配、别名与生存期会进入分析，编译器随后还要证明它们可以被删掉。反之，过晚缓冲化也会掩盖片上容量、地址空间和数据搬运等硬约束。最佳位置不是固定的“尽可能晚”，而是：先完成依赖语义的结构变换，再在存储决策开始影响合法性与成本之前显式化内存。

这就是信息寿命 (information lifetime) 的工程权衡。高层信息保留越久，通用融合和调度的选择越多；但抽象层越多，接口、规范化规则和转换路径的维护面也越大。低层细节暴露越早，后端越容易落实 ABI 与硬件约束；但许多结构一旦粉碎，就只能通过不可靠的模式识别重建。多层 IR 的价值不在“层数多”，而在让每类事实恰好活到最后一个需要它的决策点。

8.5 从 MLIR 到 LLVM，再到特定设备

MLIR 的 `llvm` 方言是对应 LLVM IR 语义的一种 MLIR 方言，并不等同于外部 LLVM IR 文件。通常先把模块完全合法化为可翻译的 LLVM 及目标内在方言，再机械地翻译为 LLVM IR；排序 `memref` 在这里落实为描述符，调用约定和地址空间也必须具体化 [28,29]。两套表示不是竞争关系：MLIR 在领域结构和硬件选择尚未确定时保存信息，LLVM 在这些选择已经具体化后承担成熟的低层优化与代码生成。

异构设备使这条原则更鲜明。AscendNPU IR 在 MLIR 上定义 HIFusion、HIVM 与 HACC 等方言：前者面向结构化融合，HIVM 描述 tile、搬运、同步与片上存储，HACC 连接宿主和设备入口 [14,16]。但官方 `VecAdd` 示例的起点已经是 HIVM/`memref` 层：两个 GM 输入经 `hivm.hir.load` 搬入 UB，执行 `hivm.hir.vadd`，再由 `hivm.hir.store` 写回 GM [15]。它具体说明了设备边界上的地址空间与数据运动，却不代表任意 `linalg` 程序都能自动穿过 HIFusion 到达该层。

对截断点积，一个自然的设计是让映射、限幅和归约先保持为结构化操作；随后依据目标向量宽度与 UB 容量形成 tile，将 GM-UB 搬运、局部归约和尾部条件显式化；最后才落实设备入口与宿主调用。这里最难的并非把乘法翻译成某条指令，而是决定部分和放在哪里、怎样合并，以及搬运和同步是否抵消并行收益。官方 `VecAdd` 恰好标出一条诚实的知识边界：它展示“低层设备程序怎样写”，没有替我们完成“高层归约怎样调度”。

因此，多层 IR 给出的不是自动优化承诺，而是一种组织决策的办法：结构仍有用时不急于丢弃；存储开始支配可行性时显式缓冲化；越过 ABI 或设备边界前，用合法性谓词消除所有未兑现的抽象。编译器由此不再是一串按名称排列的 Pass，而是一系列有意识的信息降解。

9 前沿：四组无法被彻底消除的张力

编译技术的前沿并不是一列等待安装的新功能。验证编译、超优化、PGO、JIT、异构调度和多层 IR 之所以持续演进，是因为它们分别在几组不可同时取满的目标之间移动边界。设候选编译方案为 x ，我们可以用一个帕累托（Pareto）问题概括这些选择：

$$\text{ParetoMin}_{x \in \mathcal{X}, x \models \mathcal{S}} (C_{\text{run}}(x), C_{\text{compile}}(x), C_{\text{memory}}(x), C_{\text{maint}}(x), R_{\text{semantic}}(x)). \quad (21)$$

$\mathcal{X}$ 是可考虑的实现与调度， $\mathcal{S}$ 是必须满足的语义和平台约束；各分量分别代表运行、编译、内存、维护成本与残余语义风险。现实系统很少存在支配所有其他方案的唯一最优点。所谓进步，常常是扩展可行集、降低某项成本，或让原本隐含的风险可以度量。

9.1 保证范围与适用覆盖

验证编译器希望对一族输入程序证明语义保持。若源程序为 S 、目标程序为 T ，其基本形态是行为细化（behavioral refinement）

$$T = C(S) \implies \text{Beh}(T) \subseteq \text{Beh}(S), \quad (22)$$

但 Beh 如何处理未定义行为、I/O、发散和并发，本身就是定理的一部分。CompCert 通过较受控的语言与流水线获得跨多道 Pass 的机器证明，并缩小可信计算基（trusted computing base）[23,8,53]；Alive2 则选择翻译验证（translation validation），逐次检查 LLVM IR 优化前后的细化关系，从而更容易接入快速演化的工业编译器，但受到操作覆盖、循环界和求解预算限制 [51]。

二者揭示的不是“证明优于测试”这样简单的层级，而是保证范围与生态覆盖之间的交换。模型越完整、可信计算基越小，支持新语言特征和工具链组件的成本通常越高；覆盖越广、演化越快，就越需要局部验证器、测试和工程约束共同承担信任。

本文案例中的对应决策是把关键许可压缩为一个容易检查的不变量： $0 \leq k \leq 8$ 且每项限幅到 $[-8, 12]$ ，因此整数归约不会溢出。我们不需要证明整条工具链，便能说明为什么加法重结合在这个域内合法；若未来去掉长度上界，则必须撤回这项优化许可。前沿方向是让这种局部事实更容易穿过 IR，被重写系统和验证器共同消费，而不是把每个课程例子都包装成端到端形式化工程。

9.2 搜索空间与编译预算

经典优化器常以贪心次序应用规则，速度可控，却可能因为早期选择而错过更好的组合。多面体模型（polyhedral model）把仿射循环的迭代域、访问和依赖变成整数约束，使交换、融合、分块与并行化成为调度搜索 [3,36]；超优化（superoptimization）直接搜索短程序空间 [62,61]；等式饱和（equality saturation）用等价图（e-graph）同时保存许多等价表达式，最后按成本抽取，从而推迟破坏性的规则次序 [64,4]。

候选越多并不自动意味着结果越好。搜索需要燃料上限，等价规则需要适用前提，成本模型还可能把指令数较少误判为真实机器上更快。工业系统因而倾向

于缩小搜索域：只对无副作用区域饱和，只综合短整数片段，或离线发现规则后交给独立检查器。它们没有消灭组合爆炸，只是把预算花在收益密度最高的地方。

截断点积把这项张力变得具体。逐项的 $\min/\max$ 、比较与选择可以在受限 e -图中选择分支式或无分支式实现；外层归约可比较标量链、向量水平归约与展开树。但候选集合必须携带前节的不溢出条件，而且优化目标要针对 RV64GC 或实际 NPU，而不是抽象“操作数最少”。我们的设计决策因此是：在结构化 IR 中延迟循环形态，保留若干调度选择；进入后端后收窄空间，以目标成本和编译时限作出确定选择。

9.3 静态知识与运行时知识

静态编译器知道类型、控制依赖和目标 ISA，却不知道部署时最常出现的输入、真正的热路径或动态类型。配置文件引导优化 (profile-guided optimization, PGO) 把运行剖面带回下一次构建，BOLT 在最终二进制布局已经可见时重排代码，机器学习引导优化 (machine-learning-guided optimization) 尝试从历史程序学习复杂启发式 [27,6,54,63,30]。即时编译 (just-in-time compilation, JIT) 则把决策继续推迟到运行期，以编译开销、预热和代码缓存换取真实类型与热度；分层运行时在快速基线代码与优化代码之间进一步折中 [1,35]。

这里没有脱离工作负载的“最佳编译时刻”。若截断点积只在短命进程中调用一次，JIT 的准备成本不可能由八次乘法偿还；若它成为长寿命服务中的热点，且 k 几乎总为 8，运行时版本化可以把边界处理移出热路径。可是一旦输入分布改变，旧 profile 和学习模型就会成为过期的编译输入。静态证明决定候选是否允许，运行信息决定候选是否值得；前者不能由热点数据替代，后者也不能从语义定理推导。

本案例选择预先编译 (ahead-of-time compilation, AOT) 路径，是因为目标是解释从源程序到进程映像的组成，而非追求一个八元素内核的峰值性能。这个选择本身体现了式 (21)：牺牲运行时特化机会，换取更小的系统状态、更直接的 ABI 和更清晰的观察路径。若场景改变，合理答案也应改变，而不是把 AOT 或 JIT 升格为原则。

9.4 抽象能力与互操作边界

高层抽象保存形状、归约和布局，使融合、自动调度与跨设备映射成为可能；低层接口稳定、工具丰富，也更容易同既有运行时、链接器和调试器互操作。MLIR、TVM、Halide 与 Tiramisu 等系统从不同角度分离计算含义与调度选择 [22,7,55,2]。代价是每增加一种方言、布局或调度层，就增加一组语义接口、转换规则与失败诊断。抽象若没有明确的消去边界，最终只会把复杂性推给下游。

第 8 节中的点积路线因此做了两次有意的收缩。第一次在结构化层保留映射-限幅-归约，使 CPU 向量化或 NPU 分块仍有选择；第二次在选定目标后落实 memref、地址空间、ABI 和设备入口，使剩余系统能够依赖普通链接与运行时协议。Ascend 的官方 VecAdd 位于第二次收缩之后：它清楚展示 GM-UB 搬运和 HIVM 向量操作，却不承担高层归约调度。把这条边界说清，比画一条貌似全自动的端到端箭头更有价值。

这种张力也解释了为何全程序优化通常采用摘要与选择性导入，而不是永久维持一个巨大全局 IR。ThinLTO 用模块摘要换取跨翻译单元可见性，同时保留

并行后端与缓存能力 [18,38]; Transform dialect 把部分调度策略显式表示出来, 却仍需目标成本和可诊断的失败语义 [52]。成功的抽象不是永不泄漏, 而是在泄漏发生时拥有窄而明确的接口。

9.5 共同方向：使选择及其条件显式化

四组张力最终汇合到同一个设计观：编译器的职责不是找到脱离条件的“最佳程序”，而是在可承担的知识、时间和信任预算下，选择一个满足语义约束的实现。形式化方法让允许什么更清楚；搜索方法扩大可以选择什么；profile 与 JIT 改变何时知道什么；多层 IR 决定哪些信息能活到选择发生的时刻。

对贯穿全文的截断点积，我们可以据此读出四个相互连接的决定：用范围不变量许可归约变换；用结构化表示保留有限的调度空间；因程序短小而选择 AOT，不引入无法摊销的动态状态；在 LLVM、ELF 和设备 ABI 边界主动消去高层抽象。换一个数值域、工作负载或硬件，这些决定都可能变化，但推理框架不变。

这也是“深入理解编译系统”比罗列 Pass 更重要的地方。编译器的每一层都在交换信息：丢掉一种自由，换来一种确定性；暴露一种机器事实，失去一种高层对称；扩大一种搜索能力，支付一种验证与维护成本。前沿研究真正改变设计空间之处，不是又增加一个术语，而是让这些交换可表达、可组合，并让工程师知道自己为何停在帕累托前沿上的这一点。

10 结论：阅读编译系统的三条原则

本文从一个只有几十行的 SysY 截断点积出发，追踪同一项计算如何不断改变形状：宏常量消失在记号流中，表达式获得名字绑定和类型，循环成为控制流图中的回边与静态单赋值形式 (Static Single Assignment, SSA) 中的递归值，数组下标成为地址运算，函数调用成为寄存器约定、符号和重定位，结构化归约又可以在多层中间表示 (Multi-Level Intermediate Representation, MLIR) 中延后展开。贯穿这些变化的并不是某种共同的文本形状，而是各层之间对程序意义的承接关系。由此可以提炼出三条彼此依赖的原则。

1. **意义是关系，表示是选择。**相邻表示不必相像：源语言中的循环、LLVM IR 中的基本块、RV64 中的分支序列可以实现同一个状态变换。语义关系划定哪些转换是合法的，表示选择决定哪些事实容易被分析，而成本模型才在合法候选中决定采用哪一个。把三者混为一谈，就会把“能够证明正确”误写成“值得生成”，或把“机器上更快”误写成“语义上更正确”。本案例中的逐项截断不能移到求和之后，这是意义施加的硬约束；是否展开循环、使用分支还是条件选择、是否保留压缩指令空间，则取决于代码尺寸、吞吐量、编译时间和目标微架构。正确性决定可行域，工程成本决定落点。
2. **表示管理着信息的寿命，也管理着承诺的时机。**每一种中间表示都像一份信息预算：带类型抽象语法树保留源级结构和诊断位置，SSA 暴露定义-使用关系，机器表示显式化寄存器类别和指令约束，ELF 则保留尚未求值的符号关系。降低不是把同一内容换一种拼写，而是逐步提交决定，并常常不可逆地丢弃结构。过早把归约展开成地址与跳转，会让后续阶段重新猜测并行性；无限期保留高层抽象，又会扩大语义接口、合法化规则与维护成本。因此更稳健的原则不是“层次越高越好”，而是让一项信息活到最后一个能够利用

它的阶段，随后明确地将它变为较低层的约束。MLIR 的方言共存把这种信息寿命从偶然的流水线顺序提升为可设计的系统结构。

3. **局部最优必须服从跨边界协作。**单个函数若独占整个世界，可以自由选择寄存器、布局和调用序列；真实系统却要求不同语言、编译单元、库与装载环境共同工作。应用二进制接口（Application Binary Interface, ABI）牺牲一部分局部自由，以换取独立编译；符号和重定位推迟地址承诺，以换取归档复用、布局选择和装载灵活性；链接时优化和即时编译则用更多全局信息换取更长的构建时间、更复杂的部署边界或运行期开销。这里不存在无代价的“全局最优”：可见范围越大，潜在优化越多，但缓存、增量构建、模块自治和生态兼容性也越难维持。编译系统的成熟之处，正在于知道哪些决定值得跨越边界协调，哪些决定必须由稳定接口隔离。

这三条原则把形式理论与工程现实连接起来。形式语义、控制流图、不动点分析、活跃性和重定位方程提供了可以推理的骨架；启发式搜索、分阶段编译、稳定接口和渐进式降低则回答如何在有限时间、有限内存和长期维护中实现它。理论不是现实编译器的简化替身，工程也不是对理论的不严谨妥协：前者说明不可越过的边界，后者决定在边界之内把复杂性放在哪里。

因此，阅读一个陌生编译系统时，不妨从一个具体对象开始：指出一个记号、AST 节点、SSA 值、机器指令、符号或重定位，问它表示哪项程序意义，在什么关系下承接上一层，显式保存了哪些事实，又已经提交了哪些决定；继而追问，这个决定是由正确性强制的，还是由成本模型偏好的，它又为独立编译、目标适配或后续优化留下了多少自由。能够回答这些问题，便不再只是沿着流水线背诵阶段名称，而是在阅读一套关于意义、信息与选择的计算系统。

References

1. Android Open Source Project: ART Just-in-Time Compiler, <https://source.android.com/docs/core/runtime/jit-compiler>, accessed 2026-09-22
2. Baghdadi, R., Ray, J., Romdhane, M.B., Sozzo, E.D., Akkas, A., Zhang, Y., Suriana, P., Kamil, S., Amarasinghe, S.: Tiramisu: A polyhedral compiler for expressing fast and portable code. In: 2019 IEEE/ACM International Symposium on Code Generation and Optimization. pp. 193–205 (2019). <https://doi.org/10.1109/CGO.2019.8661197>
3. Bondhugula, U., Hartono, A., Ramanujam, J., Sadayappan, P.: A practical automatic polyhedral parallelizer and locality optimizer. In: Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 101–113 (2008). <https://doi.org/10.1145/1375581.1375595>
4. Bytecode Alliance: Cranelift e-graph optimizer rfc (2023), <https://github.com/bytecodealliance/rfcs/blob/main/accepted/cranelift-egraph.md>, accepted RFC; accessed 2026-09-22
5. Chaitin, G.J., Auslander, M.A., Chandra, A.K., Cocke, J., Hopkins, M.E., Markstein, P.W.: Register allocation via coloring. *Computer Languages* **6**(1), 47–57 (1981). [https://doi.org/10.1016/0096-0551\(81\)90048-5](https://doi.org/10.1016/0096-0551(81)90048-5)
6. Chen, D., Li, D.X., Moseley, T.: AutoFDO: Automatic feedback-directed optimization for warehouse-scale applications. In: 2016 IEEE/ACM International Symposium on Code Generation and Optimization. pp. 12–23 (2016). <https://doi.org/10.1145/2854038.2854044>

7. Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., Guestrin, C., Krishnamurthy, A.: TVM: An automated end-to-end optimizing compiler for deep learning. In: 13th USENIX Symposium on Operating Systems Design and Implementation. pp. 578–594 (2018), <https://www.usenix.org/conference/osdi18/presentation/chen>
8. CompCert Project: The CompCert Verified Compiler, <https://compcert.org/>, accessed 2026-09-22
9. Cytron, R., Ferrante, J., Rosen, B.K., Wegman, M.N., Zadeck, F.K.: Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems* **13**(4), 451–490 (1991). <https://doi.org/10.1145/115372.115320>
10. Free Software Foundation: GNU Binary Utilities: ar (2026), <https://sourceware.org/binutils/docs/binutils/ar.html>
11. Free Software Foundation: The GNU Linker (2026), <https://sourceware.org/binutils/docs/ld.html>
12. Free Software Foundation: The GNU Linker: Linker Scripts (2026), <https://sourceware.org/binutils/docs/ld/Scripts.html>
13. GNU Project: The C Preprocessor: Initial Processing, <https://gcc.gnu.org/onlinedocs/cpp/Initial-processing.html>, accessed 2026-09-22
14. Huawei Ascend Community: AscendNPU IR Architecture Design. Huawei (2026), <https://ascendnpu-ir.gitcode.com/en/introduction/architecture.html>, accessed 2026-09-22; source revision 8d49415f4d75d53dc983353e49b6c99004b0f17c
15. Huawei Ascend Community: AscendNPU IR Compilation and Execution Example. Huawei (2026), https://ascendnpu-ir.gitcode.com/en/introduction/quick_start/examples.html, accessed 2026-09-22; low-level HIVM VecAdd and CANN host
16. Huawei Ascend Community: AscendNPU-IR source repository (2026), <https://github.com/Ascend/AscendNPU-IR/tree/8d49415f4d75d53dc983353e49b6c99004b0f17c>, apache-2.0; revision 8d49415f4d75d53dc983353e49b6c99004b0f17c
17. ISO/IEC JTC1/SC22/WG14: Programming languages—c: Committee draft n1570. Tech. Rep. N1570, International Organization for Standardization (2011), <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>
18. Johnson, T., Amini, M., Li, X.D.: ThinLTO: Scalable and incremental LTO. In: 2017 IEEE/ACM International Symposium on Code Generation and Optimization. pp. 111–121 (2017). <https://doi.org/10.1109/CGO.2017.7863733>
19. Kerrisk, M., man-pages contributors, L.: `execve(2)` — Linux Manual Page. Linux man-pages project (2025), <https://man7.org/linux/man-pages/man2/execve.2.html>, linux man-pages 6.15
20. Lattner, C., Adve, V.: LLVM: A compilation framework for lifelong program analysis and transformation. In: Proceedings of the 2004 International Symposium on Code Generation and Optimization. pp. 75–86. IEEE Computer Society (2004). <https://doi.org/10.1109/CGO.2004.1281665>
21. Lattner, C., Adve, V.: LLVM: A compilation framework for lifelong program analysis and transformation. In: Proceedings of the International Symposium on Code Generation and Optimization. pp. 75–86 (2004). <https://doi.org/10.1109/CGO.2004.1281665>
22. Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J.A., Riddle, R., Shpeisman, T., Vasilache, N., Zinenko, O.: MLIR: Scaling compiler infrastructure for domain specific computation. In: 2021 IEEE/ACM International

- Symposium on Code Generation and Optimization (CGO). pp. 2–14 (2021). <https://doi.org/10.1109/CGO51591.2021.9370308>
23. Leroy, X.: Formal verification of a realistic compiler. *Communications of the ACM* **52**(7), 107–115 (2009). <https://doi.org/10.1145/1538788.1538814>
 24. LLVM Project: Clang Command Line Argument Reference and Command Guide, <https://clang.llvm.org/docs/CommandGuide/clang.html>, accessed 2026-09-22
 25. LLVM Project: Clang Compiler Internals Manual, <https://clang.llvm.org/docs/InternalsManual.html>, accessed 2026-09-22
 26. LLVM Project: Clang Driver Internals, <https://clang.llvm.org/docs/DriverInternals.html>, accessed 2026-09-22
 27. LLVM Project: Clang User's Manual: Profile Guided Optimization, <https://clang.llvm.org/docs/UsersManual.html#profile-guided-optimization>, accessed 2026-09-22
 28. LLVM Project: LLVM Dialect, <https://mlir.llvm.org/docs/Dialects/LLVM/>, accessed 2026-09-22
 29. LLVM Project: LLVM IR Target, <https://mlir.llvm.org/docs/TargetLLVMIR/>, accessed 2026-09-22
 30. LLVM Project: Machine Learning Guided Optimization, <https://llvm.org/docs/MLGO.html>, accessed 2026-09-22
 31. LLVM Project: MLIR Bufferization, <https://mlir.llvm.org/docs/Bufferization/>, accessed 2026-09-22
 32. LLVM Project: MLIR Dialect Conversion, <https://mlir.llvm.org/docs/DialectConversion/>, accessed 2026-09-22
 33. LLVM Project: MLIR Interfaces, <https://mlir.llvm.org/docs/Interfaces/>, accessed 2026-09-22
 34. LLVM Project: MLIR Language Reference, <https://mlir.llvm.org/docs/LangRef/>, accessed 2026-09-22
 35. LLVM Project: ORCv2 Design and Implementation, <https://llvm.org/docs/ORCv2.html>, accessed 2026-09-22
 36. LLVM Project: Polly: Polyhedral Optimizations for LLVM, <https://polly.llvm.org/>, accessed 2026-09-22
 37. LLVM Project: Rationale for the Linalg Dialect, <https://mlir.llvm.org/docs/Rationale/RationaleLinalgDialect/>, accessed 2026-09-22
 38. LLVM Project: ThinLTO, <https://clang.llvm.org/docs/ThinLTO.html>, accessed 2026-09-22
 39. LLVM Project: Alias Analysis (2026), <https://llvm.org/docs/AliasAnalysis.html>, accessed 2026-09-22
 40. LLVM Project: Auto-Vectorization in LLVM (2026), <https://llvm.org/docs/Vectorizers.html>, accessed 2026-09-22
 41. LLVM Project: Data Layout (2026), <https://llvm.org/docs/LangRef.html#data-layout>, accessed 2026-09-22
 42. LLVM Project: Global Instruction Selection (2026), <https://llvm.org/docs/GlobalISel/>, accessed 2026-09-22
 43. LLVM Project: GlobalISel: Core Pipeline (2026), <https://llvm.org/docs/GlobalISel/Pipeline.html>, accessed 2026-09-22
 44. LLVM Project: LLVM Language Reference Manual (2026), <https://llvm.org/docs/LangRef.html>, accessed 2026-09-22
 45. LLVM Project: The LLVM Target-Independent Code Generator (2026), <https://llvm.org/docs/CodeGenerator.html>, accessed 2026-09-22
 46. LLVM Project: LLVM's Analysis and Transform Passes (2026), <https://llvm.org/docs/Passes.html>, accessed 2026-09-22

47. LLVM Project: The Often Misunderstood GEP Instruction (2026), <https://llvm.org/docs/GetElementPtr.html>, accessed 2026-09-22
48. LLVM Project: Opaque Pointers (2026), <https://llvm.org/docs/OpaquePointers.html>, accessed 2026-09-22
49. LLVM Project: Using the New Pass Manager (2026), <https://llvm.org/docs/NewPassManager.html>, accessed 2026-09-22
50. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. pp. 65–79 (2021). <https://doi.org/10.1145/3453483.3454030>
51. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for llvm. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. pp. 65–79 (2021). <https://doi.org/10.1145/3453483.3454030>
52. Lücke, M.P., Zinenko, O., Moses, W.S., Steuwer, M., Cohen, A.: The MLIR transform dialect: Your compiler is more powerful than you think. In: 2025 IEEE/ACM International Symposium on Code Generation and Optimization. pp. 241–254 (2025). <https://doi.org/10.1145/3696443.3708922>
53. Monniaux, D., Boulmé, S.: The trusted computing base of the compcert verified compiler. In: Programming Languages and Systems (ESOP). pp. 204–233 (2022). https://doi.org/10.1007/978-3-030-99336-8_8
54. Panchenko, M., Auler, R., Nell, B., Ottoni, G.: BOLT: A practical binary optimizer for data centers and beyond. In: 2019 IEEE/ACM International Symposium on Code Generation and Optimization. pp. 2–14 (2019). <https://doi.org/10.1109/CGO.2019.8661201>
55. Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., Amarasinghe, S.: Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 519–530 (2013). <https://doi.org/10.1145/2491956.2462176>
56. RISC-V International: RISC-V ABIs Specification (Nov 2022), <https://docs.riscv.org/reference/abi/v1.0/index.html>, ratified specification
57. RISC-V International: RISC-V ABIs Specification: Linker Relaxation (2026), <https://docs.riscv.org/reference/abi/riscv-elf-linker-relaxation.html>, living specification, accessed 2026-09-22
58. RISC-V International: RISC-V Assembly Programmer’s Manual (2026), <https://github.com/riscv-non-isa/riscv-asm-manual>, living upstream manual, accessed 2026-09-22
59. RISC-V International: The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (Jan 2026), https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf, release 20260120
60. RISC-V International: RISC-V ISA Extension Naming Conventions (2026), <https://docs.riscv.org/reference/isa/unpriv/naming.html>, unprivileged architecture specification, accessed 2026-09-22
61. Sasnauskas, R., Chen, Y., Collingbourne, P., Ketema, J., Taneja, J., Regehr, J.: Souper: A synthesizing superoptimizer. arXiv preprint arXiv:1711.04422 (2017), <https://arxiv.org/abs/1711.04422>
62. Schkufza, E., Sharma, R., Aiken, A.: Stochastic superoptimization. In: Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 305–316 (2013). <https://doi.org/10.1145/2451116.2451150>

63. Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., Li, D.: MLGO: A machine learning guided compiler optimizations framework. arXiv preprint arXiv:2101.04808 (2021), <https://arxiv.org/abs/2101.04808>
64. Willsey, M., Nandi, C., Wang, Y.R., Flatt, O., Tatlock, Z., Panthekha, P.: egg: Fast and extensible equality saturation. Proceedings of the ACM on Programming Languages **5**(POPL) (2021). <https://doi.org/10.1145/3434304>
65. Xnuos: System V Application Binary Interface: Dynamic Linking (2025), <https://gabi.xnuos.com/elf/08-dynamic.html>
66. Xnuos: System V Application Binary Interface: ELF Object File Format (2025), <https://gabi.xnuos.com/elf.pdf>
67. Zhao, J., Nagarakatte, S., Martin, M.M.K., Zdancewic, S.: Formalizing the LLVM intermediate representation for verified program transformations. In: Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. pp. 427–440 (2012). <https://doi.org/10.1145/2103656.2103709>

A GitHub 仓库

本小组的 GitHub 仓库链接如下：

`https://github.com/hard-male-alliance/principles-of-compiler-systems`